# STAY AND SHOP

## DOCUMENTATION

Nolan Schirripa

Peter Ingalsbe

Christine Seng

Georgia Rushing

Karter Sanamo

Matthew Peterson

# CHANGE LOG

| | |
|---|---|
| 02/18/2026 | Added use cases |
| 02/18/2026 | Added domain model |
| 02/18/2026 | Added SSDs |
| 02/18/2026 | Added operation contracts |

# USE CASE DIAGRAM

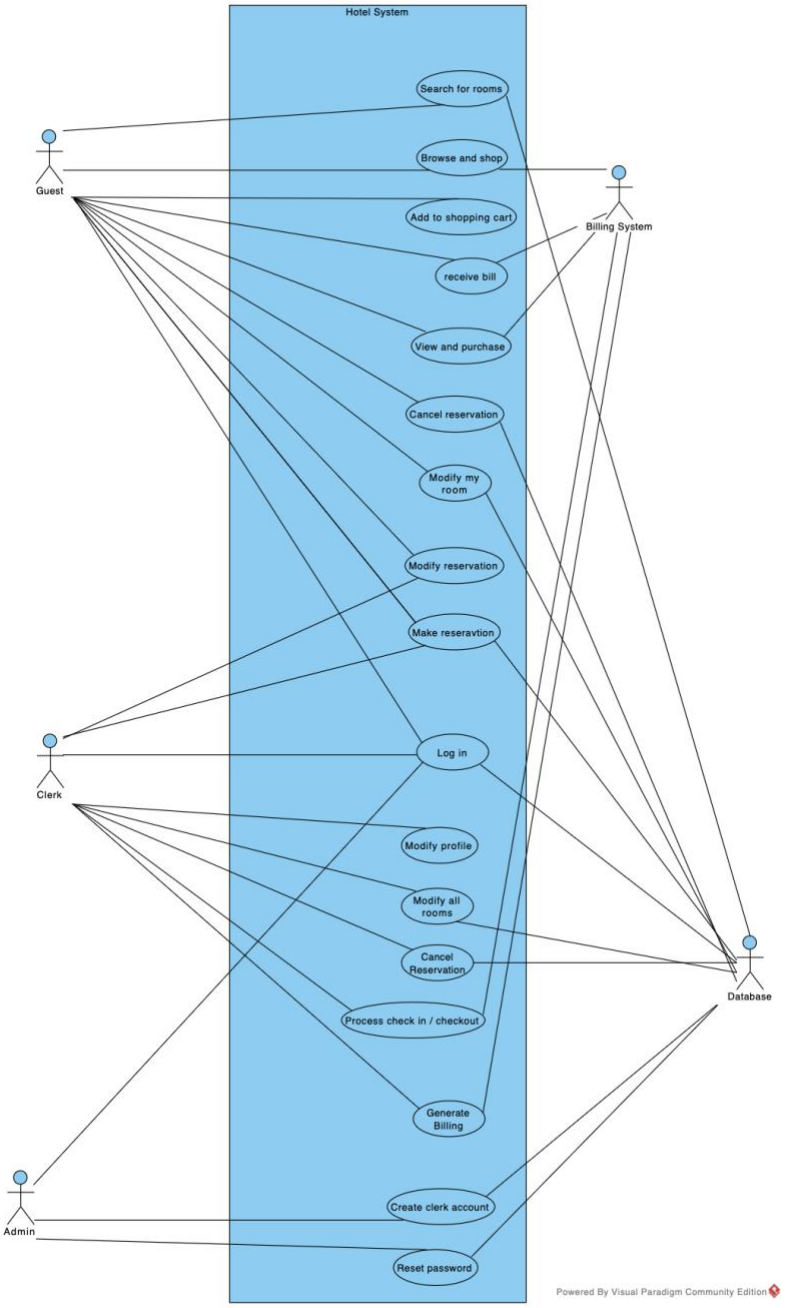



**Figure 1: Use case diagram**

# USE CASES

## ACCOUNT LOG IN AND CREATION

1. **Name: Create guest account** (by Nolan Schirripa)
    - Actor: Guest, Cradmin
    - Main Success Scenario:
        1. User navigates to page and selects option to create new account
        2. The system prompts user to enter criteria for valid username and password
        3. User enters username and password
        4. System validates input
        5. System displays message confirming account has been created successfully
    - Preconditions:
        1. The hotel system is online and operational
        2. Guest account data exists in the system
    - Postconditions:
        1. Guest account is stored in the system
        2. No existing accounts are modified
    - Extensions (alternative flows)
        - 4a.) Invalid username and password
            1. System detects if password does not meet requirements
            2. System detects if username already being used
            3. Error message displayed
    - Special requirements:

        - System should adapt to creation of account within reasonable time (1-2 seconds)
        - Displayed messages must be consistent with current data

2. **Name: Create clerk account** (Peter Ingalsbe)
    - Actor(s): Admin
    - Preconditions:
        1. Admin is signed in
        2. System is online
    - Postconditions:
        1. New clerk account is created
        2. Other accounts are unchanged
    - Main success Scenario:
        1. New clerk info is entered
        2. New account is created in system
    - Extensions:
        - 1a.) Input info is invalid
            1. Displays error message and prompts to reinput info
    - Special Requirements: N/A

3. **Name: Log in** (by Georgia Rushing)
    - Actor: Guest, Clerk, Admin
    - Preconditions:
        1. The hotel system is online and operational
        2. User has account in the system
    - Postconditions:
        1. User is granted access to the system
    - Main Success Scenario:
        1. User selects to "Log In"
        2. System prompts user for their username and password
        3. User enters valid input
        4. System authenticates input and grants access
        5. System displays correct home page
    - Extensions:

- 4a) Invalid User Input
    1. System detects invalid input (username or password)
    2. System displays error message and requests corrected input
- 5a) Missing User input
    1. User submits login without username or password
    2. System detects missing input
    3. System displays error message and requests missing input

- Special Requirements:
    - Authentication should happen within reasonable time (1-2 seconds)
    - System logs login attempts

4. **Name: Reset Password** (by Matthew Peterson)
    - Actor(s): Admin
    - Preconditions:
        1. User resetting password is actively logged in as Admin
        2. Targeted user's email/username is known
        3. Targeted user has existing account in the system
    - Postconditions:
        1. User's password is updated and changed successfully
        2. User can login with new password
        3. Old password no longer works
    - Main Success Scenario:
        1. Admin logs into system with administrator credentials
        2. Admin selects option to manage user accounts
        3. System displays a searchable list of user accounts
        4. Admin selects a user account
        5. Admin selects to reset user's password
        6. System prompts a confirmation to reset the user's password

7. Admin confirms the reset
8. System prompts admin for the new password
9. System updates user account with the new password
10. System displays confirmation message regarding the reset

- Extensions:
  - 4a.) User account does not exist
    1. System would not show the user in the list of users
  - 6a.) Admin cancels reset at confirmation message
    1. System cancels entire operation
    2. No changes are made to the account
- Special Requirements:
  - System should log all password reset attempts
  - Password reset should complete within a reasonable amount of time (1-2 seconds)

5. **Name: Modify Profile** (by Georgia Rushing)
   - Actor(s): Clerk
   - Preconditions:
     1. The hotel system is online and operational
     2. Clerk has successfully logged into system
   - Postconditions:
     1. Clerk's profile information updated
   - Main Success Scenario:
     1. Clerk selects their profile
     2. System displays current profile information
     3. Clerk edits the information to be modified
     4. System validates input
     5. System updates profile information
     6. System displays confirmation

- Extensions:
    - 4a) Invalid input
        1. System deletes invalid input
        2. System displays error message
        3. System displays prompt for information to be re-entered
- Special Requirements:
    - Displayed profile information should be consistent with updated profile information

## RESERVATIONS AND ROOM INFORMATION

1. **Name: Check in guest** (by Christine Seng)
    1. Actor: Clerk
    2. Main Success Scenario:
        1. Clerk selects option to check in a guest
        2. Clerk searches for associated reservation with either guest name or reservation confirmation number
        3. System validates input
        4. System displays a list of all possible reservations that match the provided information and the current date
        5. Clerk selects the appropriate reservation to check in the guest with
        6. System marks the Reservation as occupied and displays confirmation to the Clerk
    3. Preconditions:
        1. The System is online and operational
        2. User is logged in to a Clerk account
        3. Previously created Reservations are accessible by the System
    4. Postconditions:

1. No Rooms/Reservations created/deleted
2. The only Reservation modified is the one the guest is checked in for, where status is changed to indicate the guest has been checked in

5. Extensions (alternative flows):
    - 3a) Clerk does not enter valid log in information
        1. System returns to the log in page and re-prompts Clerk for valid input
    - 4a) Guest name or confirmation has no associated reservations
        1. Report the error and return to the previous page to search again
    - 6a) If a guest has a current reservation for a room that is occupied
        1. Report an error to the Clerk
6. Special Requirements:
    - Search input should be validated in a reasonable amount of time (1-2 secs)
    - System should display search results in a reasonable amount of time (1-2 secs)
    - Displayed reservations should be consistent with current reservations in the system

2. **Name: Make Reservation** (by Christine Seng)
    - Actor(s): Guest, Clerk (making reservation on behalf of a Guest)
    - Main Success Scenario:
        1. Guest/Clerk selects option to make a reservation
        2. System prompts guest/clerk to enter criteria for their desired room
        3. Guest/Clerk enters criteria
        4. System validates input
        5. System searches for rooms available for the specified criteria
        6. System displays a list of available rooms that match the criteria with types and price

7. Guest/Clerk selects a room
8. Guest/Clerk enters all relevant guest information
9. System validates input
10. System saves guest information and the room being reserved
11. System marks the specified room as reserved for the specified dates
12. System displays a message saying the room has successfully been reserved

- Preconditions:

  1. System is online and operational
  2. Room and reservation data exist in system

- Postconditions:

  1. A new Reservation with appropriate information created
  2. No other rooms reserved or other reservations modified

- Extension (alternative flows):

  - 4a) Invalid search criteria

    1. System deletes invalid input
    2. System displays error message
    3. System displays prompt for information to be re-entered

  - 5a) No rooms available

    1. System displays message saying no rooms are available for specified criteria, gives user option to exit or enter new criteria

  - 9a) Invalid guest information

    1. System deletes invalid input
    2. System displays error message
    3. System displays prompt for information to be re-entered

- Special requirements:

1. System should display search results within reasonable time (1-2 secs)
2. Displayed information should be consistent with current reservations
3. Guest information should be validated within reasonable time (1-2 secs)

3. **Name: Cancel guest reservation** (by Christine Seng)

- Actor: Clerk
- Main Success Scenario:
    1. Clerk selects option to cancel a reservation
    2. System prompts Clerk for guest name
    3. Clerk enters guest name
    4. System validates input
    5. System searches for guest and displays a list of reservations with matching names associated
    6. Clerk selects appropriate reservation to cancel
    7. System cancels the reservation and indicates to the clerk that the reservation has been successfully cancelled
- Preconditions:
    1. The System is online and operational
    2. User is logged in to a Clerk account
    3. Previously created Reservations are accessible by the System
- Postconditions:
    1. Any cancelled Reservations no longer exist in the system
    2. Rooms previously associated with cancelled Reservations are available for other reservations
    3. No other Reservations or Rooms created/modified
- Extensions (alternative flows):
    - 4a) Invalid input is detected

        1. System prompts the Clerk again for the guest's name
    - 5a) The guest does not have any reservations associated with their name
        1. System reports this and returns to the page to search for a name.
    - 5b) The reservation the guest is searching for is not displayed/in the system
        1. Clerk can navigate back to the search page without having to cancel any reservations
- Special Requirements:
    1. Search input should be validated in a reasonable amount of time (1-2 secs)
    2. System should display search results in a reasonable amount of time (1-2 secs)
    3. Displayed reservations should be consistent with current reservations in the system

4. **Name: Modify Reservation** (by Georgia Rushing)
    - Actor(s): Guest, Clerk
    - Preconditions:
        1. The hotel system is online and operational
        2. User has successfully logged into system
        3. A valid reservation exists in the system
        4. Current date is before the reservation start date
    - Postconditions:
        1. Reservation updated
        2. Updated confirmation displayed
    - Main Success Scenario:
        1. User selects to view reservations
        2. System displays list of reservations

3. User selects reservation to modify
4. System displays reservation details to edit
5. User enters new booking information
6. System validates input
7. System checks availability
8. System updates reservation
9. System displays confirmation with updated reservation details

- Extensions:
    1. 6a) Invalid search criteria
        1. System deletes invalid input
        2. System displays error message
        3. System displays prompt for information to be re-entered
    2. 7a) No rooms available
        1. System displays message saying no rooms are available for specified criteria, gives user option to exit or enter new criteria
    3. 3a) Reservation cannot be modified
        1. System displays message saying reservation cannot be modified
- Special Requirements:
    1. Displayed information should be consistent with current reservation data
    2. Validation and availability should be validated within reasonable time (1-2 secs)
    3. Any penalties must be displayed before confirmation

5. **Name: Search Available Rooms** (by Matthew Peterson)

- Actor(s): Guest
- Preconditions
    1. Hotel shop and system are online and operational

2. Guest has access to search interface (website, app, kiosk)
3. Guest provides a valid date range (check-in and check-out)

- Postconditions
    1. A list of available room types is displayed for requested dates
    2. Each result shows key details (price, beds, amenities, etc)
    3. Search results are filterable and sortable by price and amenities
- Main Success Scenario:
    1. Guest selects “Search Available Rooms”
    2. System prompts guest for check-in and check-out dates, and number of guests
    3. Guest submits criteria to system
    4. System validates information
    5. System searches through all room types to see which fit number of guests
    6. System searches through fitting room types to see which types are available for the given dates, checking existing reservations
    7. System displays available room types with details and sorting options by bed count, price, and amenities
    8. Guest can select a room to proceed to booking
        - Handoff to Make Reservation case
    9. Guest can refine or correct their search input and repeat process
- Extensions:
    - 3a.) Invalid Input
        1. System detects invalid dates or other information
        2. System displays an error message and highlights fields to fix
        3. Guest corrects input and resubmits
    - 6a.) No rooms found
        1. System finds zero matches for guests input

2. System displays no availability and suggests guest refine their search dates or guest count
3. Guest has option to search again

- Special Requirements:
  - Results should be returned in about 3-5 seconds
  - Availability must be real-time inventory
  - Prices should reflect fees and other ongoing discounts
  - Room data should just reflect available rooms, not status or guest information of reserved rooms
  - Search UI should be basic and usable on mobile, to improve navigation and limit number of keystrokes a user must take

6. **Name: View Room Status** (by Karter Sanamo)

- Actor(s): Clerk
- Preconditions:
  1. Clerk is logged in
  2. The hotel system is online and operational
  3. Room data exists in the system
- Postconditions:
  1. System logs all clerk's activity
  2. No changes to the rooms viewed in the system
- Main Success Scenario:
  1. Clerk selects the option to view room status
  2. System retrieves all rooms and their current statuses
  3. System displays a list of all rooms organized by their floor room
  4. System then displays each room's number, type, occupancy status, and reservation details

5. Clerk can select a specific room to view more details
6. The system returns the full room details including occupancy status, bed type, smoking status, quality level, and current guest (if occupied)
7. Clerk can then go back to the room list, or exit entirely

- Extensions:
  - 2a.) No rooms are in the system
    1. The system would then display a message that there are no rooms in the system
    2. Prompt the clerk with a message to add rooms to the system
- Special Requirements:
  - Room status should load within a reasonable time (<1 second)
  - Displayed room data should be accurate and consistent with the current information/reservations

7. **Name: Modify room information** (Nolan Schirripa)

- Actor: Clerk
- Main Success Scenario:
  1. Clerk selects room they want to modify
  2. Clerk selects the field they want to modify
  3. System validates modification
  4. System displays message confirming modification
- Preconditions:
  1. The hotel system is online and operational
  2. List of all rooms in hotel are successfully displayed
- Postconditions:
  1. Selected rooms fields are modified
  2. System adapts and responds to changes

- Extensions (alternative flows)
    - 3a.) Invalid modification
        1. System cannot validate change
        2. Fields are not modified
        3. Error message displayed for reason of failed change
- Special requirements:
    - System should adapt to modifications within reasonable time (1-2 seconds)

## HOTEL SHOP AND BILLING

1. **Name: Browse Store Products** (by Karter Sanamo)
    - Actor(s): Guest
    - Preconditions:
        1. The hotel shop and system are online and operational
        2. Product inventory data exists in the system
        3. Guest is logged in
        4. Guest has an active reservation or is checked in
    - Postconditions:
        1. The guest can view all of the available products without changes to cart or inventory stock.
        2. System will log all browsing activities for the session.
    - Main Succes Scenario:
        1. Guest selects option to browse the store

2. System displays product categories (clothing, accessories, local goods, etc.)
3. Guest will select a category
4. System will then display a list of products in that category with images, descriptions, and prices.
5. The guest can view the product details.
6. The system will display a list of detailed information about the product, including the available status of the product.
7. The guest can then continue to browse other categories or leave the store.

- Extensions (alternative flows):
  - 4a.) Category has no available products
    1. System will display a message to the logged in guest that there are no products available in this category.
    2. System then prompts the guest to browse other categories or exit the store.
- Special requirements:
  - Product information should load within reasonable time (1-2 seconds)
  - Product images should be clear and sized properly
  - Product availability status must be current and accurate.

2. **Name: Add items to cart** (by Nolan Schirripa)
   - Actor(s): Guest
   - Preconditions:
     1. The hotel shop and system are online and operational
     2. Item data exists in the system
     3. Guest has successfully logged in to the system
     4. Guest has successfully made a room reservation

- Postconditions:
    1. Selected items are added to the guest's cart
    2. Items not selected remain unchanged
- Main Succes Scenario:
    1. Number of items currently in the guest's cart are displayed
    2. Guest selects desired item to be added to the cart
    3. The system shows option for guest to add the item to the cart
    4. Guest adds the item to the cart
    5. System validates input
    6. Display message indicates item has been added to the guest's cart
    7. Guest's cart icon updates displaying number of items currently in cart
- Extensions (alternative flows):
    - 2a.) Desired item is unavailable
        1. The option to add item to cart appears but is marked as unavailable
        2. If the guest tries to use the cart function with this item an error message is displayed
- Special requirements:
    - System should indicate item is added to cart within reasonable time (1-2 seconds)
    - Displayed availability information must be consistent with current restrictions

3. **Name: Purchase store item** (by Peter Ingalsbe)
    - Actor(s): Guest
    - Preconditions:
        1. The cart is not empty (items are in cart)
        2. Guest is logged in
        3. Payment info is entered

- Postconditions:
    1. Store's stock is changed.
    2. Cart is emptied.
    3. Correct billing info is saved.
- Main Success Scenario:
    1. Guest enters criteria for payment
    2. System authorizes payment
    3. Confirmation message is displayed
    4. Cart items are removed
- Extensions:
    - 2a). Purchase method is invalid (ex. Credit card number is wrong)
        1. System will display a message informing which field is wrong
        2. Guest will not be charged
        3. Cart will not be emptied
- Special Requirements:
    - Billing system will check info within a reasonable timeframe
    - Useful and specific error message about invalid transaction

4. **Name: View Shopping Cart** (Karter Sanamo)
    - Actor(s): Guest
    - Preconditions:
        1. The hotel shop and system are online and operational
        2. Guest is logged in
        3. Guest has an active reservation/is checked in
        4. Guest has at minimum one item in their cart
    - Postconditions:
        1. No changes are to be made to the inventory or the order status
        2. The system must log all cart view activity for this session
    - Main success scenario:
        1. Guest selects the option to view
        2. System retrieves and displays all of the items in the cart for that guest

3. System shows every item with the name, image, quantity, price, and then the subtotal at the bottom
4. Guest can update the quantity of any item
5. System, then recalculates and updates the subtotal after the update
6. Guest can either
    - Proceed to checkout and pay for their items then
    - Continue browsing for more items (go back)

- Extensions:
    - 3a.) The item in the cart becomes unavailable/out of stock while shopping
        1. The system flags the item as "unavailable"
        2. The system notifies the guest and prompts them to remove it effectively, not letting them check out with the item, until it's removed.
- Special Requirements:
    - Cart contents should load within a reasonable time (<1 seconds)
    - Prices and availability must always reflect the current data (not batched updates on refreshes)
    - The overall UI should be very user-friendly and easy to navigate
    - The quantity updates should reflect immediately (not batched updates on refreshes)

5. **Name: Generate Billing Information** (by Matthew Peterson)
    - Actor(s): Hotel Clerk
    - Preconditions
        1. Hotel shop and system are online and operational
        2. Previous charges recorded in system
        3. Guests exist in the system
    - Postconditions
        1. Billing information is generated
        2. Billing information is combined stay/shop charges
        3. Bill is displayed to clerk and is exportable
        4. Bill generation is logged

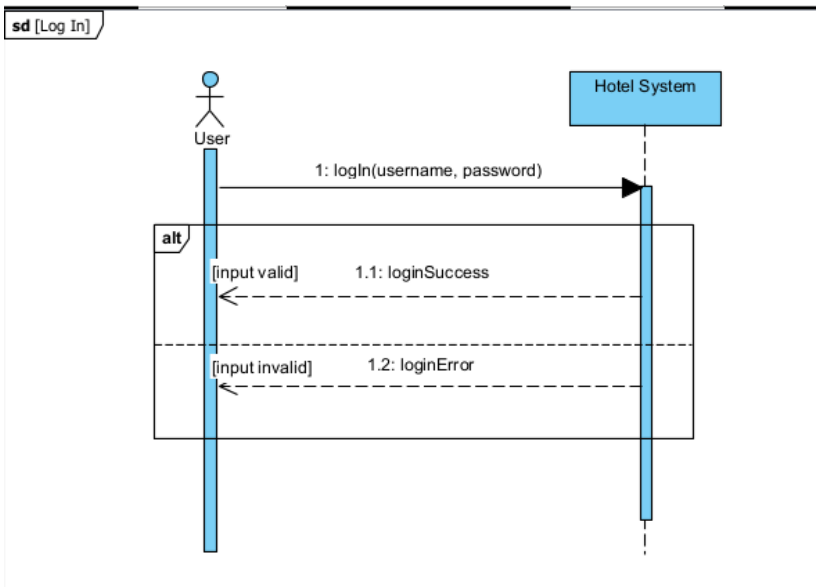
- Main Success Scenario:
    1. Clerk selects “Generate Billing Information”
    2. Enters guest lookup information (name, room, or reservation ID)
    3. Clerk searches and selects correct guest
    4. System retrieves all charges for that guest, both stay and shop
    5. System calculates totals with tax, discounts, and balance due
    6. System displays billing statement to Clerk
    7. Clerk chooses an export option (print, email, etc.)
    8. System generates a report and exports as ordered
    9. System logs all actions by Clerk
- Extensions:
    - 3a.) Guest Not Found
        1. System would return “No guest found” when lookup information is entered
        2. System would prompt clerk to enter guest information again
        3. System displays updated results if any
    - 4a.) No charges
        1. System would return “no charges found” when prompted with a given user
        2. No billing information would be generated, as there are no charges
- Special Requirements:
    - Only authorized clerk users can generate billing information
    - Billing information generated should be accurate and in a readable format
    - Response time ~3-5 seconds
    - Export options should be consistent with hotel branding

6. **Name: Receive Bill** (by Peter Ingalsbe)
    - Actor(s): Guest, billing system
    - Preconditions:
        1. The hotel shop and system are online and operational
        2. Guest has successfully logged in to the system
        3. Guest has successfully purchased an item
        4. Billing system has data on purchases for user

5. Guest is connected to internet

- Postconditions
    1. Guest can view bill document in their system
- Main Success Scenario:
    1. A list of purchases made can be seen
    2. Bill displays time of purchase and cost of each item
    3. Guests can then pay using the billing system
- Extensions (alternative flows):
    - 2a.) No purchases have been made
        1. The option to see the bill is not available
        2. If the guest is charged in error use a different system to request a refund
- Special requirements:
    - Bill should be plainly formatted and clearly spaced to allow for easy reading.

## SYSTEM SEQUENCE DIAGRAMS (SSD) AND OPERATION CONTRACTS




Log in (Georgia Rushing)

**<u>Contract</u>**: logIn

**Operation**: logIn(username, password)

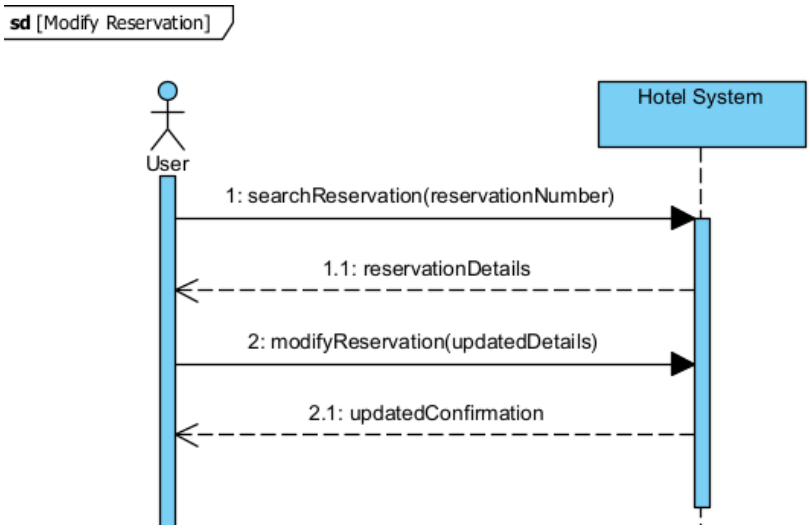
**Cross References**: Use Case—Log in

**Preconditions**:

1. The hotel system is online and operational
2. User has account in the system

**Postconditions**:

1. User is granted access to the system
2. Login status of user account is modified to authenticated
3. No user account objects created or deleted




Modify Reservation (Georgia Rushing)

**<u>Contract</u>**: modifyReservation

**Operation**: modifyReservation(reservationNumber)

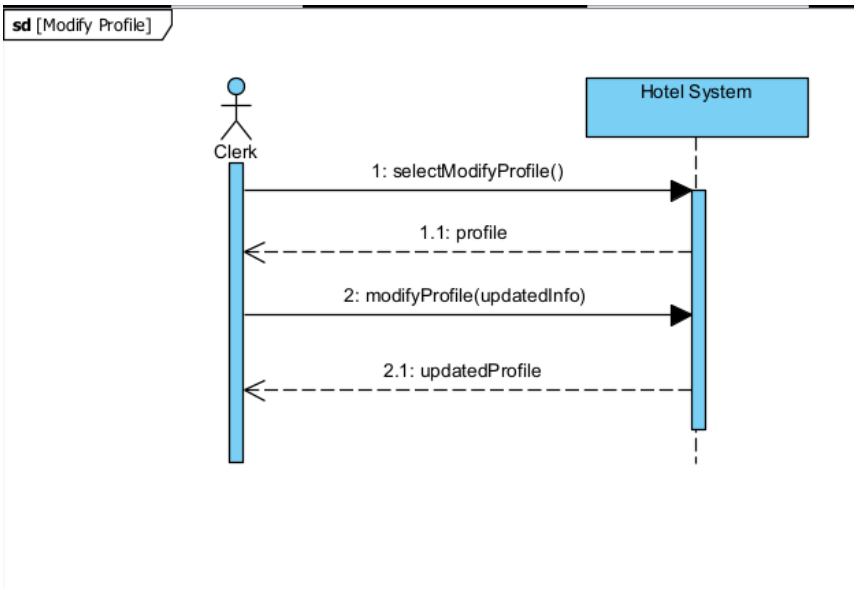
**Cross References**: Use Case—Modify Reservation

**Preconditions**:

1. The hotel system is online and operational
2. User has successfully logged into system
3. A valid reservation exists in the system
4. Current date is before the reservation start date

**Postconditions**:

1. Attributions of existing reservation updated
2. Total cost of reservation updated
3. Association between reservation and previous room removed
4. Association between reservation and new available room created
5. No new reservation objects created or deleted




Modify Profile (Georgia Rushing)

**<u>Contract</u>**: modifyProfile

**Operation**: modifyProfile(updatedInfo)

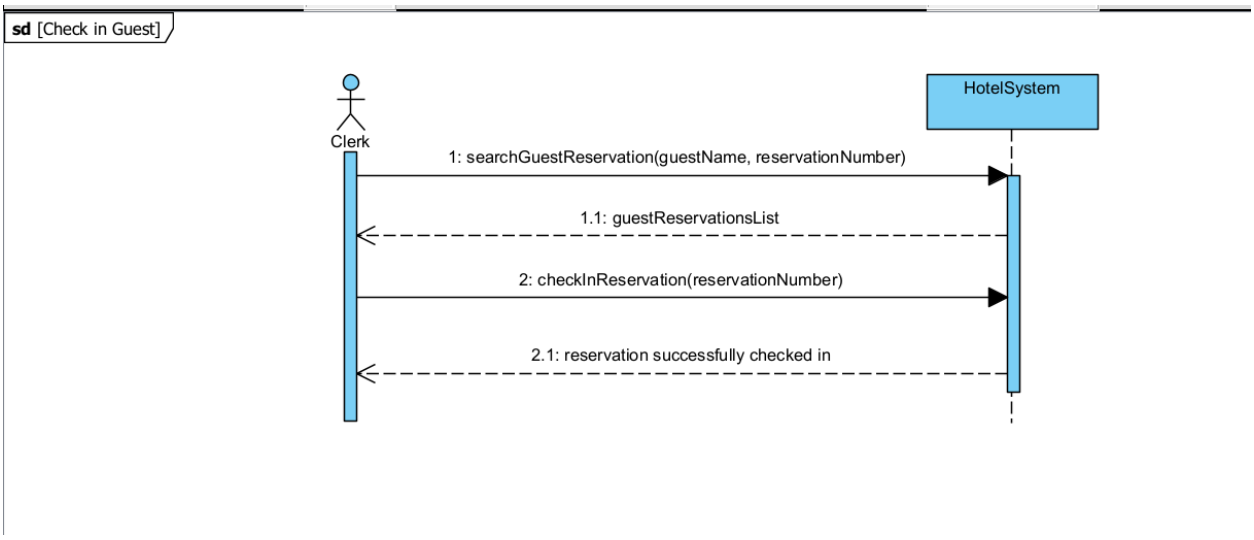
**Cross References**: Use Case—Modify Profile

**Preconditions**:

1. The hotel system is online and operational
2. Clerk has successfully logged into system

**Postconditions**:

1. Attributes of clerk's profile is changed
2. No new clerk profiles created or deleted




**Check in Guest (Christine Seng)**

**<u>Contract</u>**: checkInGuest

**Operation**: checkInGuest(reservationNumber)

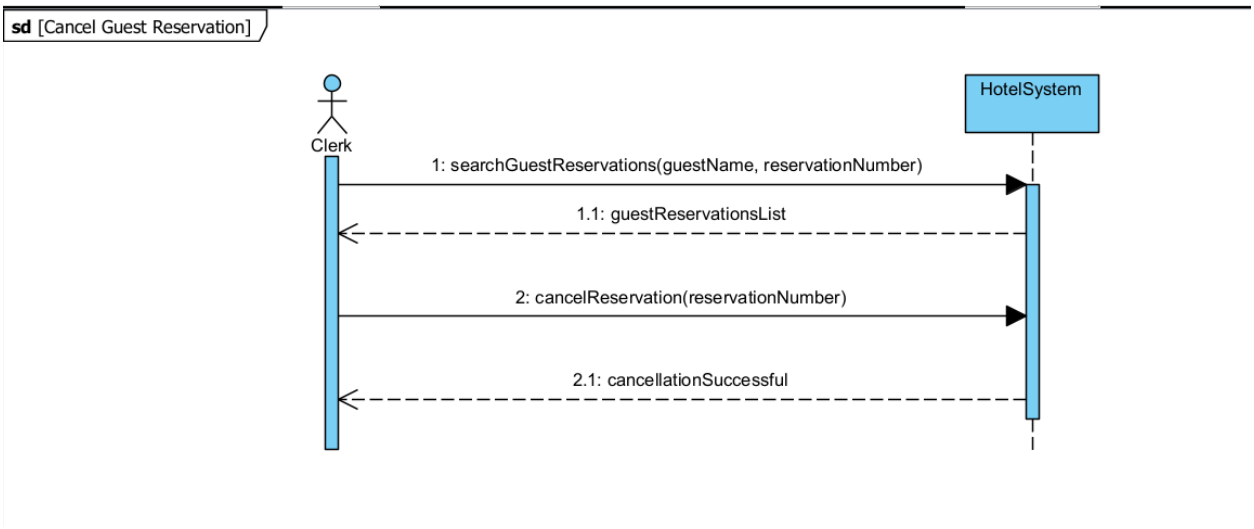
**Cross References**: Use Case—Check in Guest

**Preconditions**:

1. The hotel system is online and operational
2. Room and reservation data exist in the system

**Postconditions**:

1. Appropriate Reservation changed to indicate the guest is checked in
2. No Rooms/Reservations created or deleted
3. No other existing Rooms/Reservations modified




Cancel Guest Reservation (Christine Seng)

**Contract**: searchGuestReservations

**Operation**: searchGuestReservations(guestName, reservationNumber)

**Cross References**: Use Case—Cancel guest reservation, Check in Guest

**Preconditions**:

1. The hotel system is online and operational
2. Room and reservation data exist in the system

**Postconditions**:

1. No Rooms/Reservations created or deleted
2. No existing Rooms/Reservations modified

**Contract**: cancelReservation

**Operation**: cancelReservation(reservationNumber)

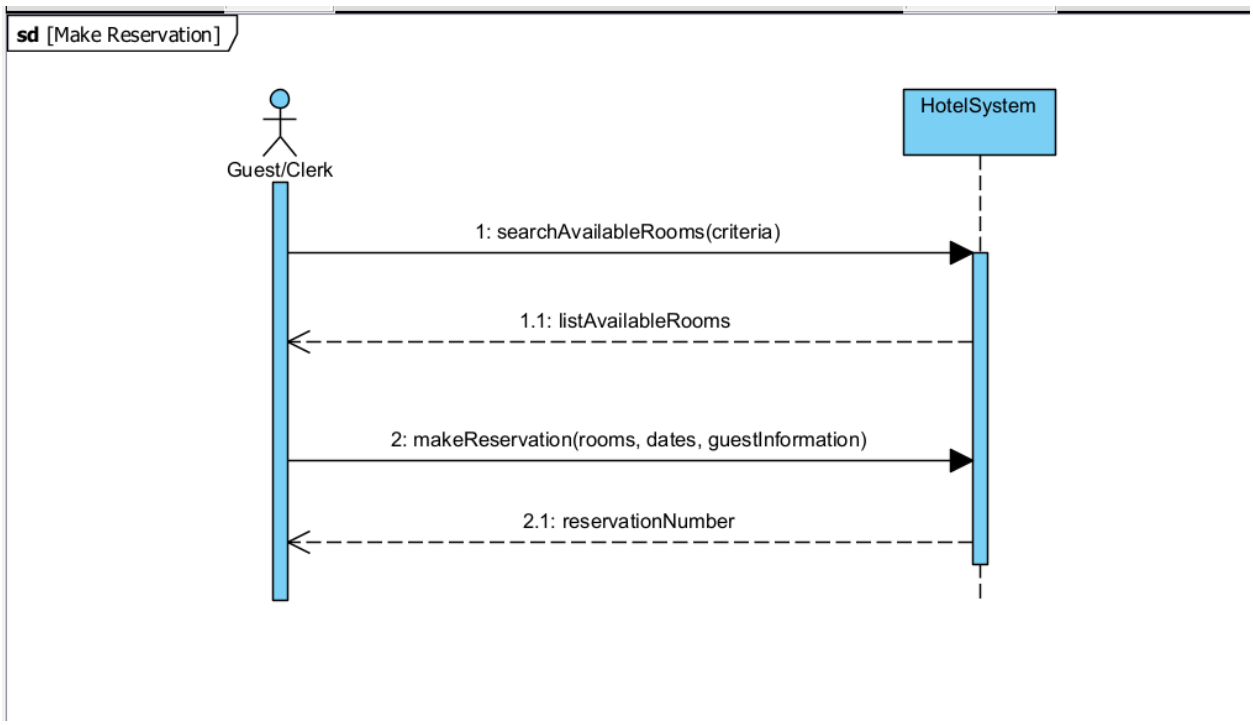
**Cross References**: Use Case—Cancel guest reservation

**Preconditions**:

1. The hotel system is online and operational
2. Room and reservation data exist in the system

**Postconditions**:

1. Appropriate Reservation deleted and no longer associated with its rooms
2. No other Rooms/Reservations created or deleted
3. No other existing Rooms/Reservations modified




Make Reservation (Christine Seng)

**Contract**: searchAvailableRooms

**Operation**: searchAvailableRooms(criteria)

**Cross References**: Use Case—Make reservation

**Preconditions**:

1. The hotel system is online and operational
2. Room and reservation data exist in the system

**Postconditions**:

1. No Rooms/Reservations created or deleted
2. No existing Rooms/Reservations modified

**Contract**: makeReservation

**Operation**: makeReservation(rooms, dates, guestInformation)

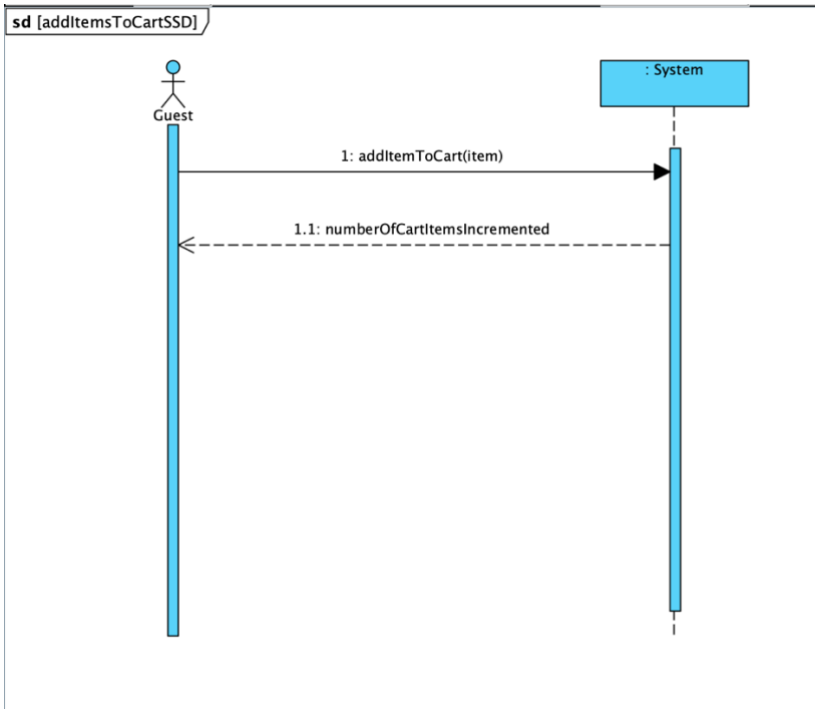
**Cross References**: Use Case—Make reservation

**Preconditions**:

1. The hotel system is online and operational
2. Room and reservation data exist in the system

**Postconditions**:

1. A new Reservation with appropriate information created
2. No other Rooms/Reservations created or deleted
3. No existing Rooms/Reservations modified




Add Items to Cart (Nolan Schirripa)

**<u>Contract</u>**: addItemToCart

**Operation**: addItemToCart(item)

**Cross References**: Use Case—Add items to cart

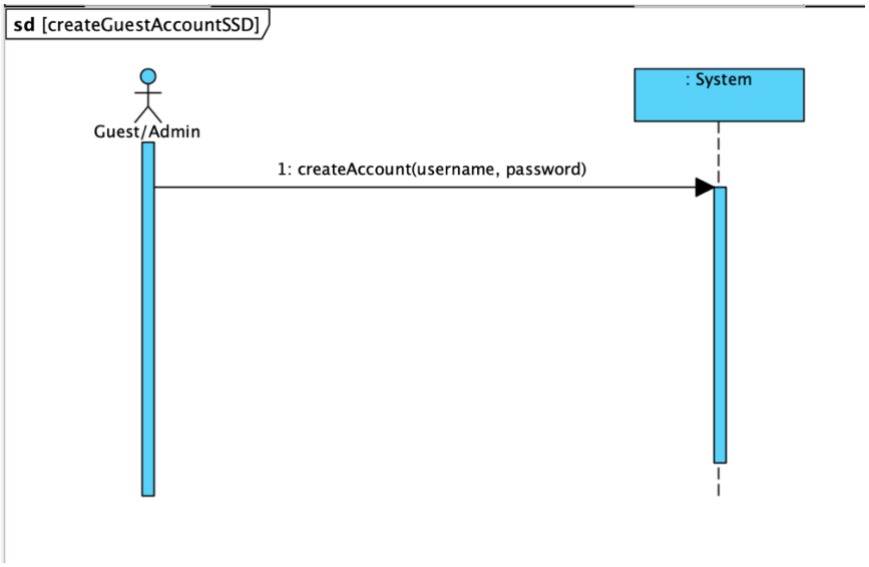
**Preconditions**:

1. The hotel shop and system are online and operational

2. Item data exists in the system
3. Guest has successfully logged into the system
4. Guest has successfully made a room reservation

**Postconditions**:

1. List of items in the cart is added to
2. No item objects are created or deleted
3. No existing items are modified




Create Guest Account (Nolan Schirripa)

**<u>Contract</u>**: createAccount

**Operation**: createAccount(username, password)

**Cross References**: Use Case—Create guest account

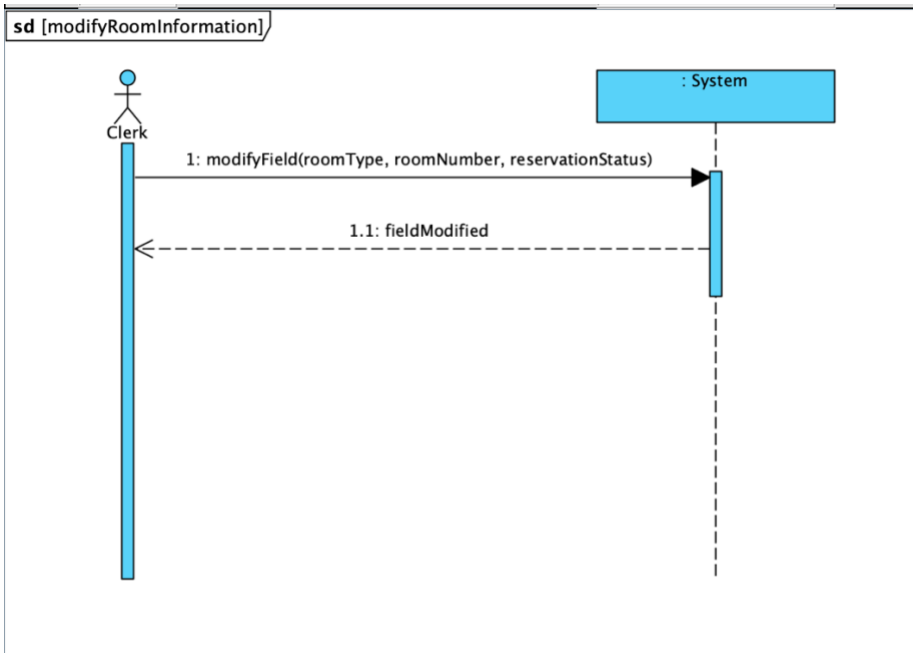
**Preconditions**:

1. The hotel system is online and operational

2. Guest account data exists in the system

**Postconditions**:

1. A guest account object is created and stored by the system
2. No existing guest accounts are modified or deleted




Modify Room Info (Nolan Schirripa)

**<u>Contract</u>**: modifyField

**Operation**: modifyField(roomType, roomNumber, reservationStatus)

**Cross References**: Use Case—Modify room information

**Preconditions**:

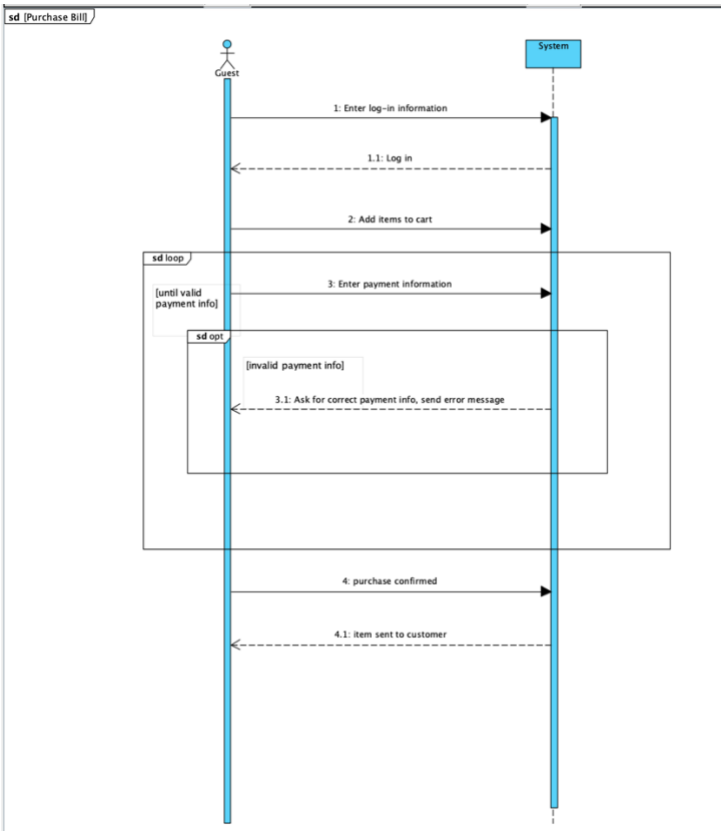
1. The hotel system is online and operational
2. List of all rooms in hotel are successfully being displayed

**Postconditions**:

1. Specified field of selected room is modified
2. Other existing rooms are not modified

3. Existing reservations are updated if necessary to show modification

4. No room objects are created or deleted




Purchase Item (Peter Ingalsbe)

**<u>Contract</u>**: logIn

**Operation**: logIn(username, password)

**Cross References**: Use Case—Log in

**Preconditions**:

3. The hotel system is online and operational

4. User has account in the system

**Postconditions**:

4. User is granted access to the system

5. Login status of user account is modified to authenticated

6. No user account objects created or deleted

**Contract**: enterPayementInfo

**Operation**: enterPayementInfo(cart, payment information)

**Cross References**: Use Case—purchase items

**Preconditions**:

1. Guest is logged in
2. Guest has items in cart
3. The hotel shop and system are online and operational

**Postconditions**:

1. Cart is unchanged
2. Store is unchanged

**Contract**: addItemToCart

**Operation**: addItemToCart(item)

**Cross References**: Use Case—purchase items

**Preconditions**:

1. Guest is logged in
2. Item is in store and in stock
3. The hotel shop and system are online and operational

**Postconditions**:

4. Cart has new item
5. Store is unchanged

**Contract**: confirmPurchase

**Operation**: confirmPurchase()

**Cross References**: Use Case—purchase items

**Preconditions**:

1. Guest is logged in
2. Guest has items in cart

3. Guest has entered payment info
4. The hotel shop and system are online and operational

**Postconditions**:

1. Cart is emptied
2. Items are removed from store
3. Bill is generated

**<u>Contract</u>**: checkPaymentInfo

**Operation**: checkPaymentInfo(paymentInfo)

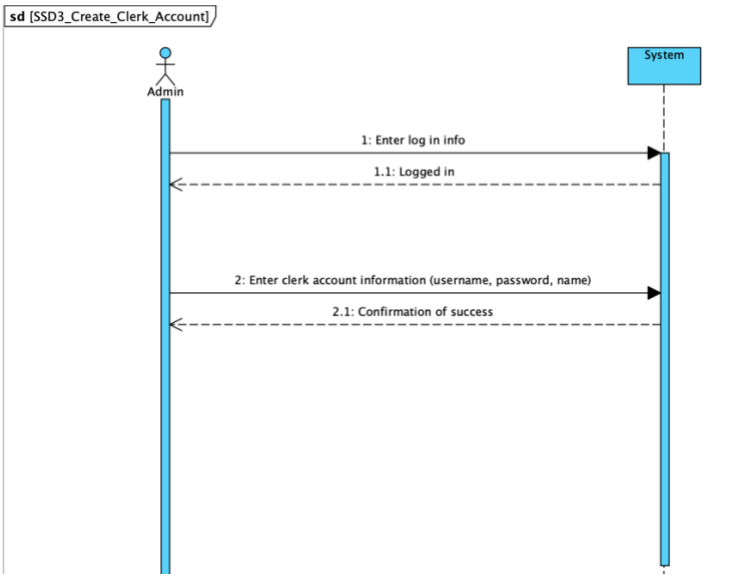
**Cross References**: Use Case—Purchase item

**Preconditions**:

1. The hotel shop and system are online and operational
2. Guest is logged in
3. Payment info is entered

**Postconditions**:

1. Cart is unmodified
2. System is unchanged




Create Clerk Account (Peter Ingalsbe)

**Contract**: logIn

**Operation**: logIn(username, password)

**Cross References**: Use Case—Log in

**Preconditions**:

5. The hotel system is online and operational
6. User has account in the system

**Postconditions**:

7. User is granted access to the system
8. Login status of user account is modified to authenticated
9. No user account objects created or deleted

**Contract**: createClerkAccount

**Operation**: createClerkAccount(username, password)

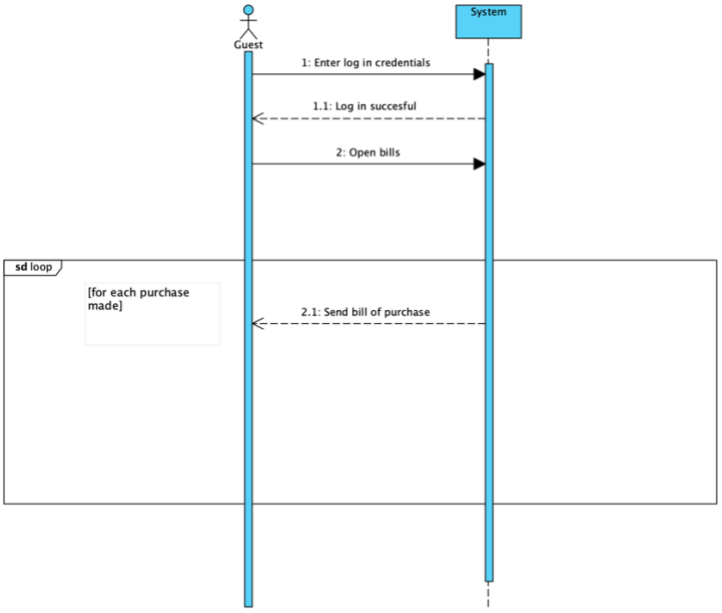
**Cross References**: Use Case—Create clerk account

**Preconditions**:

1. The hotel system is online and operational
2. Admin is logged in

**Postconditions**:

1. A clerk account object is created and stored by the system
2. No existing clerk accounts are modified or deleted




Receive Bill (Peter Ingalsbe)

**<u>Contract</u>**: logIn

**Operation**: logIn(username, password)

**Cross References**: Use Case—Log in

**Preconditions**:

7. The hotel system is online and operational
8. User has account in the system

**Postconditions**:

10. User is granted access to the system
11. Login status of user account is modified to authenticated
12. No user account objects created or deleted

**<u>Contract</u>**: openBills

**Operation**: openBills()

**Cross References**: Use Case—purchase items

**Preconditions**:

1. Guest is logged in
2. Guest has bought items
3. The hotel shop and system are online and operational

**Postconditions**:

1. System is unchanged
2. Guest has bill

**<u>Contract CO1</u>: requestPasswordReset()**

**Operation:** requestPasswordReset(userID)

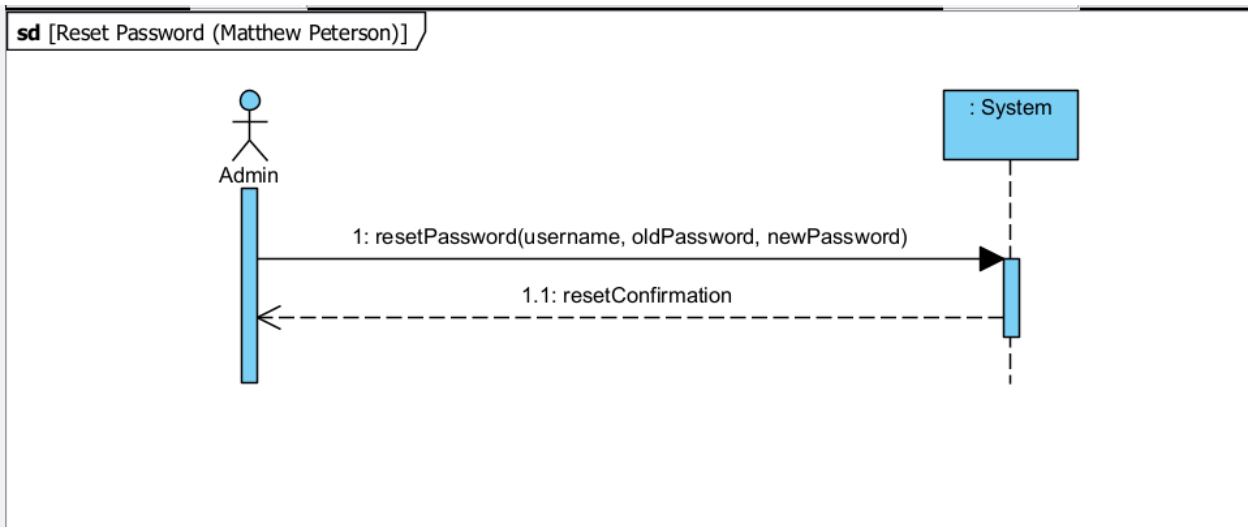
**Cross References:** Use Case – Reset Password

**Preconditions:**

1. Admin is logged into the system.
2. Admin has authorization privileges.
3. A User account with the given userID exists.

**Postconditions**:

1. No changes are made to User account *yet*.




Reset Password (Matthew Peterson)

**<u>Contract CO2</u>: confirmReset ()**

**Operation:** confirmReset ()

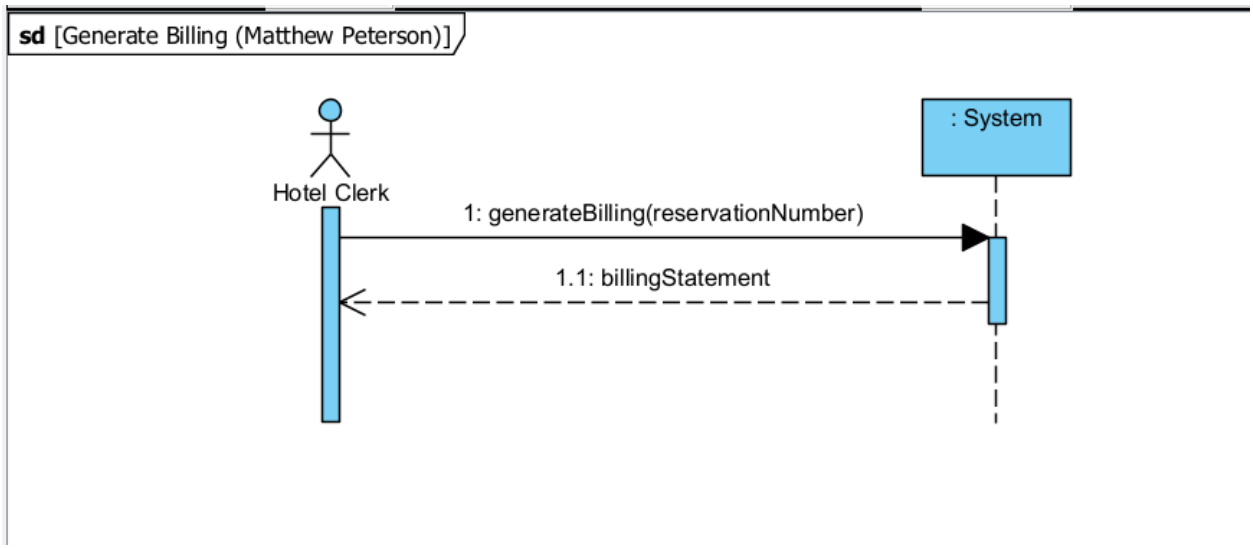
**Cross References:** Use Case – Reset Password

**Preconditions:**

1. Admin confirmed the reset request.
2. User account exists.
3. newPassword is a valid password

**Postconditions**:

1. User's password is updated to newPassword
2. Previous password no longer works.
3. PasswordResetLog object created
4. PasswordResetLog is associated with admin
5. PasswordResetLog.time is set to current date/time
6. System displays confirmation message




Generate Billing Info (Matthew Peterson)

**<u>Contract CO3</u>:** searchGuest(criteria)

**Operation:** searchGuest(criteria)

**Cross References:** Use Case – Generate Billing Information

**Preconditions:**

1. Clerk is authorized
2. System is online and operational

**Postconditions**:

1. Matching guest records are retrieved
2. If no guests found, system displays "No guests found"

**<u>Contract CO4</u>:** generateBillingStatement(guestID)

**Operation:** generateBillingStatement(reservationNumber)

**Cross References:** Use Case – Generate Billing Info

**Preconditions:**

1. Reservation exists in system
2. Clerk is authorized to generate billing info
3. System is online and operational

**Postconditions**:

1. All charge objects associated with reservation are retrieved
2. BillingStatement object created
3. BillingStatement is associated with reservation
4. BillingStatement.total is calculated
5. BillingStatement.balanceDue is calculated

**Contract CO5:** exportBilling

**Operation:** exportBilling(guestID, exportType)

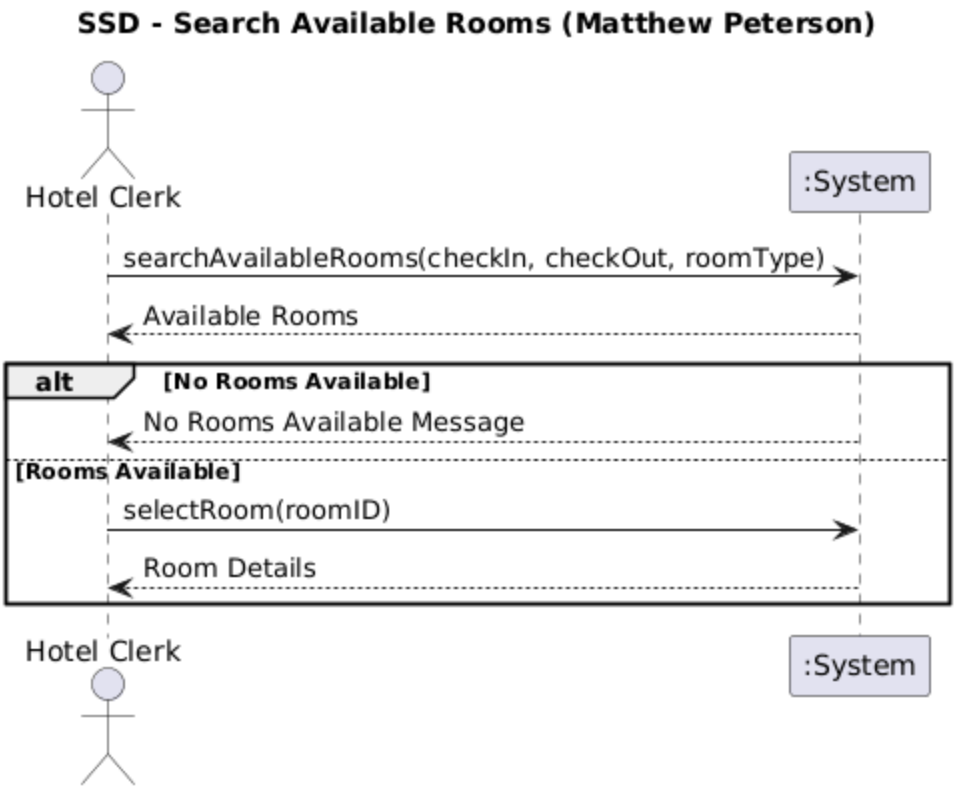
**Cross References:** Use Case – Generate Billing Info

**Preconditions:**

1. BillingStatement object exists.
2. An export option has been selected

**Postconditions**:

1. BillingStatement is formatted to hotel branding
2. BillingStatement is exported in selected format (print, email, etc)
3. BillingExportLog object created
4. BillingExportLog is associated with Clerk
5. BillingExportLog.time is set to current date/time




Search Available Rooms (Matthew Peterson)

**<u>Contract CO6</u>:** searchAvailableRooms

**Operation:** searchAvailableRooms(checkinDate, checkoutDate, guests)

**Cross References:** Use Case – Search Available Rooms

**Preconditions:**

1. checkIn and checkout are valid dates provided
2. checkinDate is prior to checkout date
3. guest is > 0
4. System is online and operational

**Postconditions**:

1. RoomType objects are filtered by capacity and guestcount
2. Available room objects identified for given date range
3. Availability is real-time inventory

**<u>Contract</u> CO7:** selectRoom

**Operation:** selectRoom(roomTypeID)

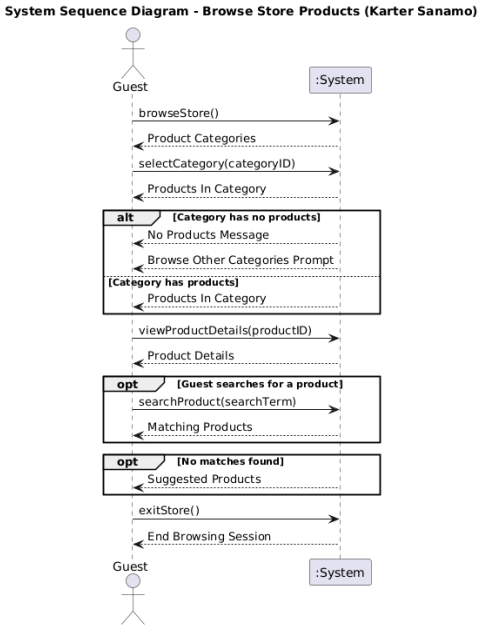
**Cross References:** Use Case – Search Available Rooms

**Preconditions:**

1. RoomType excists.
2. RoomType is available for selected dates

**Postconditions**:

1. Selected RoomType is stored temporarily
2. Control transfers to make reservation use case




**Browse Store Products (Karter Sanamo)**

**<u>Contract</u>:** browseStore()

**Operation:** browseStore()

**Cross References:** Use Case – Browse Store Products
**Preconditions:**
1. Guest is logged in
2. The shop is online

**Postconditions**:
1. The browsing session is created
2. The categories are displayed

**<u>Contract</u>:** browseStore()

**Operation:** browseStore()

**Cross References:** Use Case – Browse Store Products
**Preconditions:**

1. Guest is logged in
2. The shop is online

**Postconditions**:

1. The browsing session is created
2. Prices calculated including fees/discounts
3. If no rooms available, system has a no availability status

**Contract:** selectCategory(categoryID)

**Operation:** selectCategory()

**Cross References:** Use Case – Browse Store Products

**Preconditions:**

1. Categories displayed
2. Session is active

**Postconditions**:

1. Log entry created

**Contract:** viewProductDetails(productID)

**Operation:** viewProductDetails(productID)

**Cross References:** Use Case – Browse Store Products

**Preconditions:**

1. Products displayed
2. ProductID is valid

**Postconditions**:

1. There are no cart/inventory changes

**Contract:** searchProduct(searchTerm)

**Operation:** searchProduct(searchTerm)

**Cross References:** Use Case – Browse Store Products

**Preconditions:**

1. Session is active
2. SearchTerm is not empty

**Postconditions**:

1. Matching products are displayed
2. Suggestions are shown if there is no match

**Contract:** exitStore()

**Operation:** exitStore()

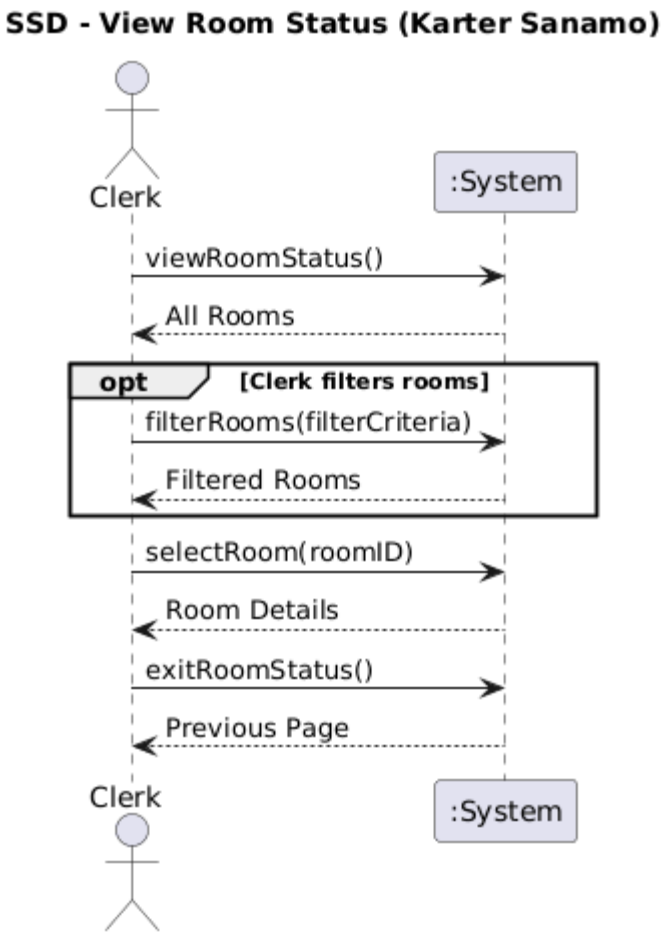
**Cross References:** Use Case – Browse Store Products

**Preconditions:**

1. Session is active

**Postconditions**:

1. End time recorded
2. Session closed




View Room Status (Karter Sanamo)

**<u>Contract</u>:** viewRoomStatus()
**Operation:** viewRoomStatus()
**Cross References:** Use Case – View Room Status
**Preconditions:**

1. Clerk is logged in
2. System is online

**Postconditions**:

1. All rooms and statuses retrieved
2. No room status changing

**<u>Contract</u>:** filterRooms(filterCriteria)
**Operation:** filterRooms(filterCriteria)
**Cross References:** Use Case – View Room Status
**Preconditions:**

1. Rooms are displayed
2. Filter criteria is valid

**Postconditions**:

1. Filtered subset of rooms

**Contract:** selectRoom(roomID)
**Operation:** selectRoom(roomID)
**Cross References:** Use Case – View Room Status
**Preconditions:**

1. Room list is displayed
2. roomID is valid

**Postconditions**:

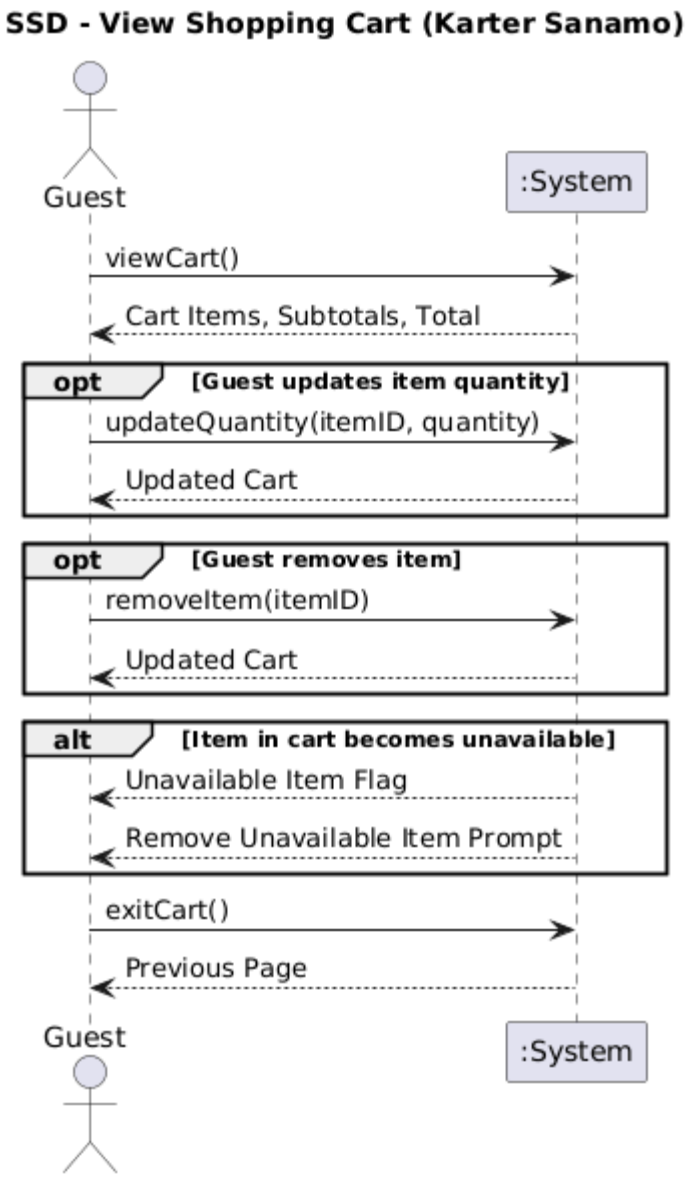
2. Full room details retrieved

**Contract:** exitRoomStatus()
**Operation:** exitRoomStatus()
**Cross References:** Use Case – View Room Status
**Preconditions:**

1. Clerk is viewing room status

**Postconditions**:

1. Session returned to previous page
2. No rooms were modified




View Shopping Cart (Karter Sanamo)

**Contract:** viewCart()
**Operation:** viewCart()
**Cross References:** Use Case – View Shopping Cart
**Preconditions:**

1. Guest is logged in
2. Cart has at least one of them

**Postconditions**:

1. Cart items, subtotals, and totals retrieved and displayed

**<u>Contract</u>:** updateQuantity(itemID, quantity)
**Operation:** updateQuantity(itemID, quantity)
**Cross References:** Use Case – View Shopping Cart
**Preconditions:**

1. Cart is displayed
2. itemID is valid
3. Quantity is greater than zero

**Postconditions**:
1. Item quantity updated
2. Cart subtotals and totals recalculated

**<u>Contract</u>:** removeItem(itemID)
**Operation:** removeItem(itemID)
**Cross References:** Use Case – View Shopping Cart
**Preconditions:**

1. ItemID is valid

**Postconditions**:
1. Item removed from cart
2. Cart total recalculates

**<u>Contract</u>:** flagUnavailableItem()
**Operation:** flagUnavailableItem()
**Cross References:** Use Case – View Shopping Cart
**Preconditions:**

1. Item availability status changed to unavailable

**Postconditions**:
1. Item flagged as unavailable
2. Guest prompted to remove item

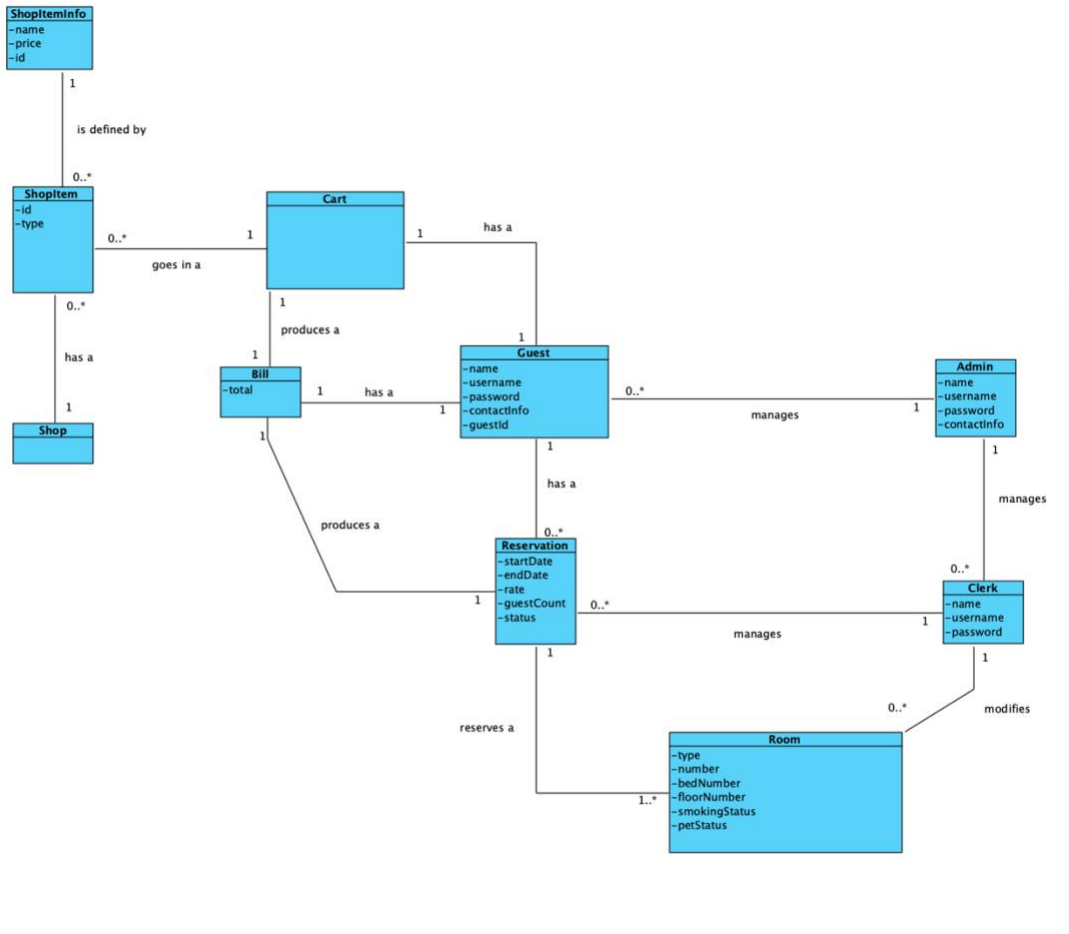
**<u>Contract</u>:** exitCart()
**Operation:** exitCart()
**Cross References:** Use Case – View Shopping Cart
**Preconditions:**

1. Guest is viewing cart

**Postconditions**:
1. Session returned to previous page

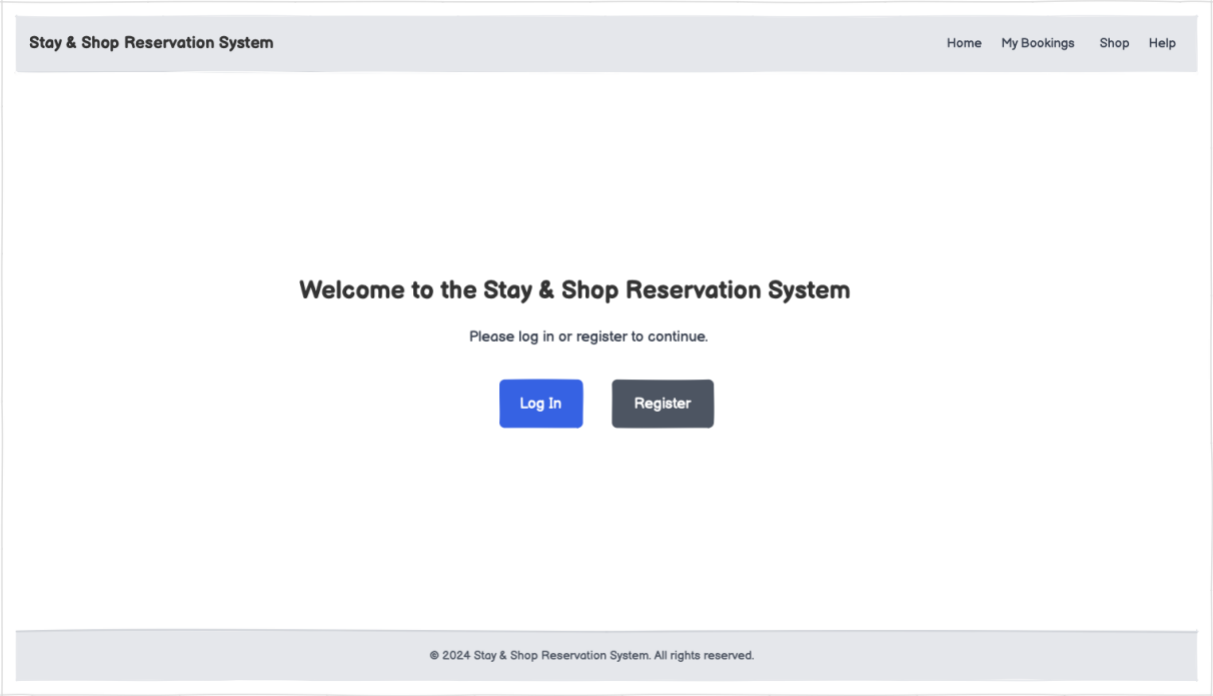
2. No inventory or order changes made

# DOMAIN MODEL

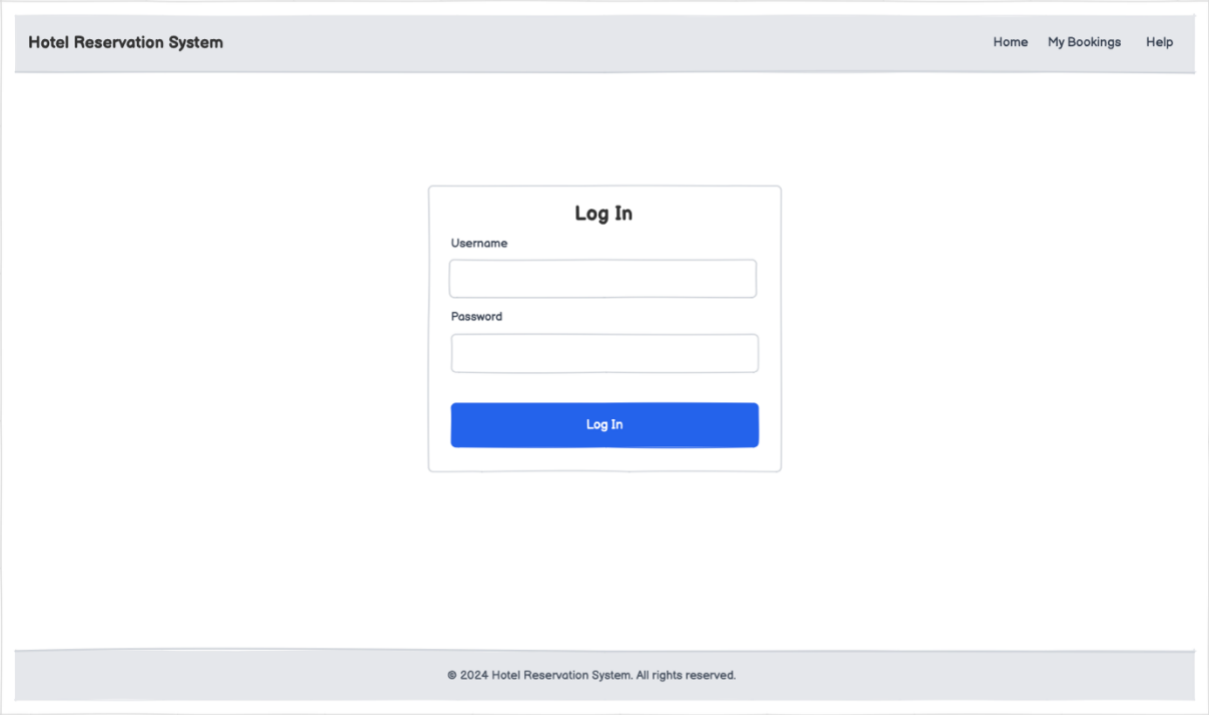



**Domain Model**

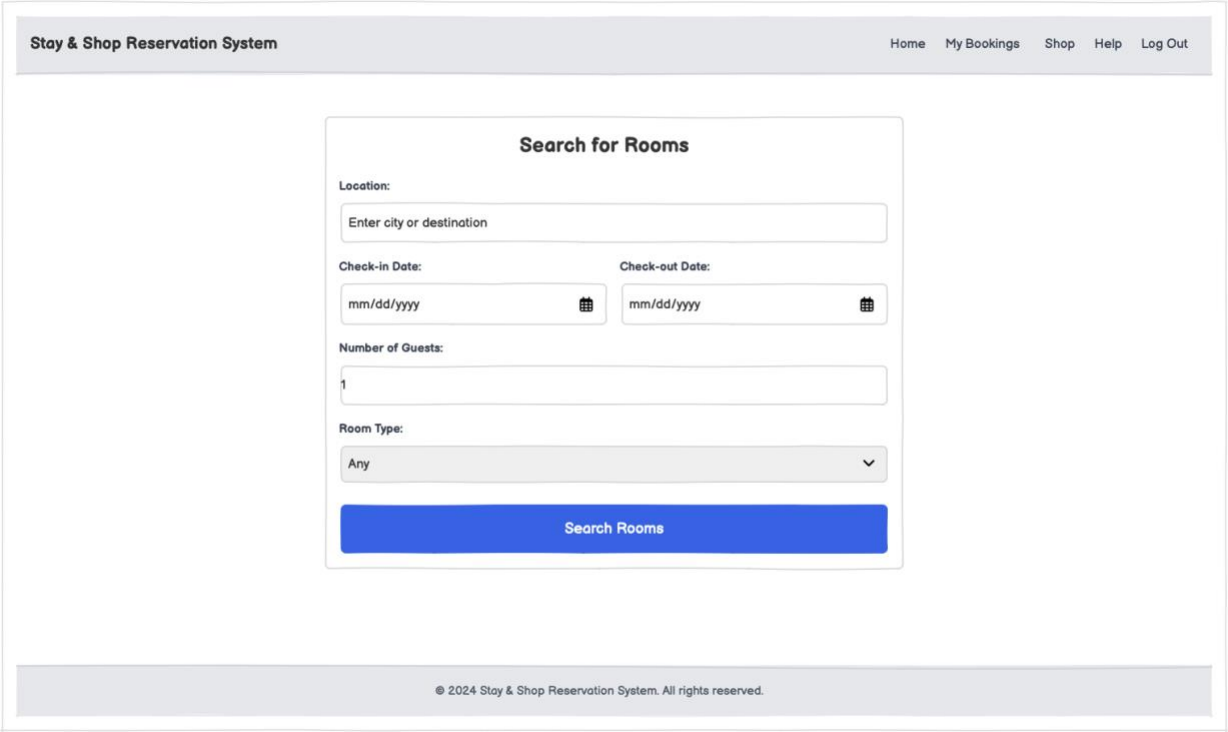
# WIREFRAMES



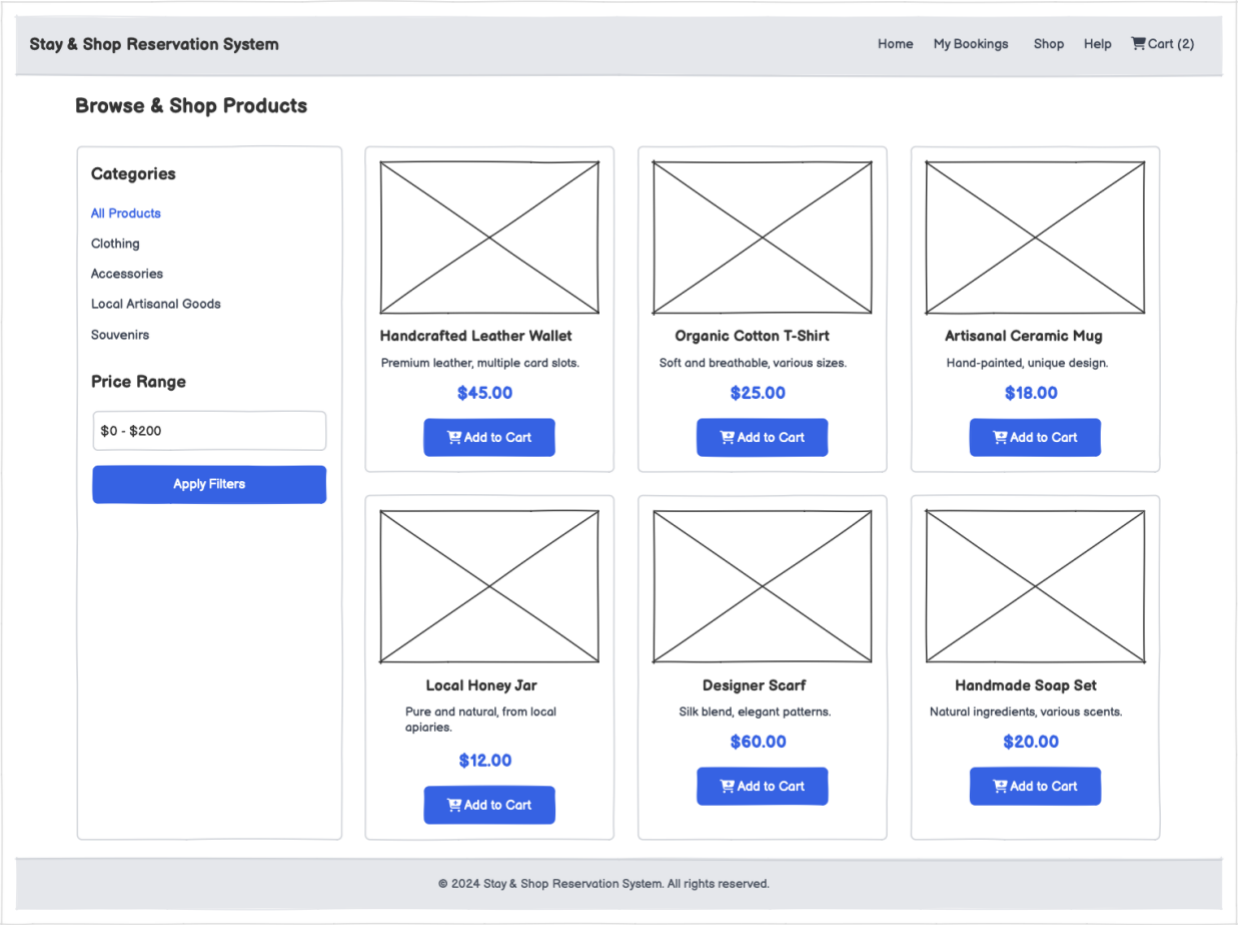

Welcome page



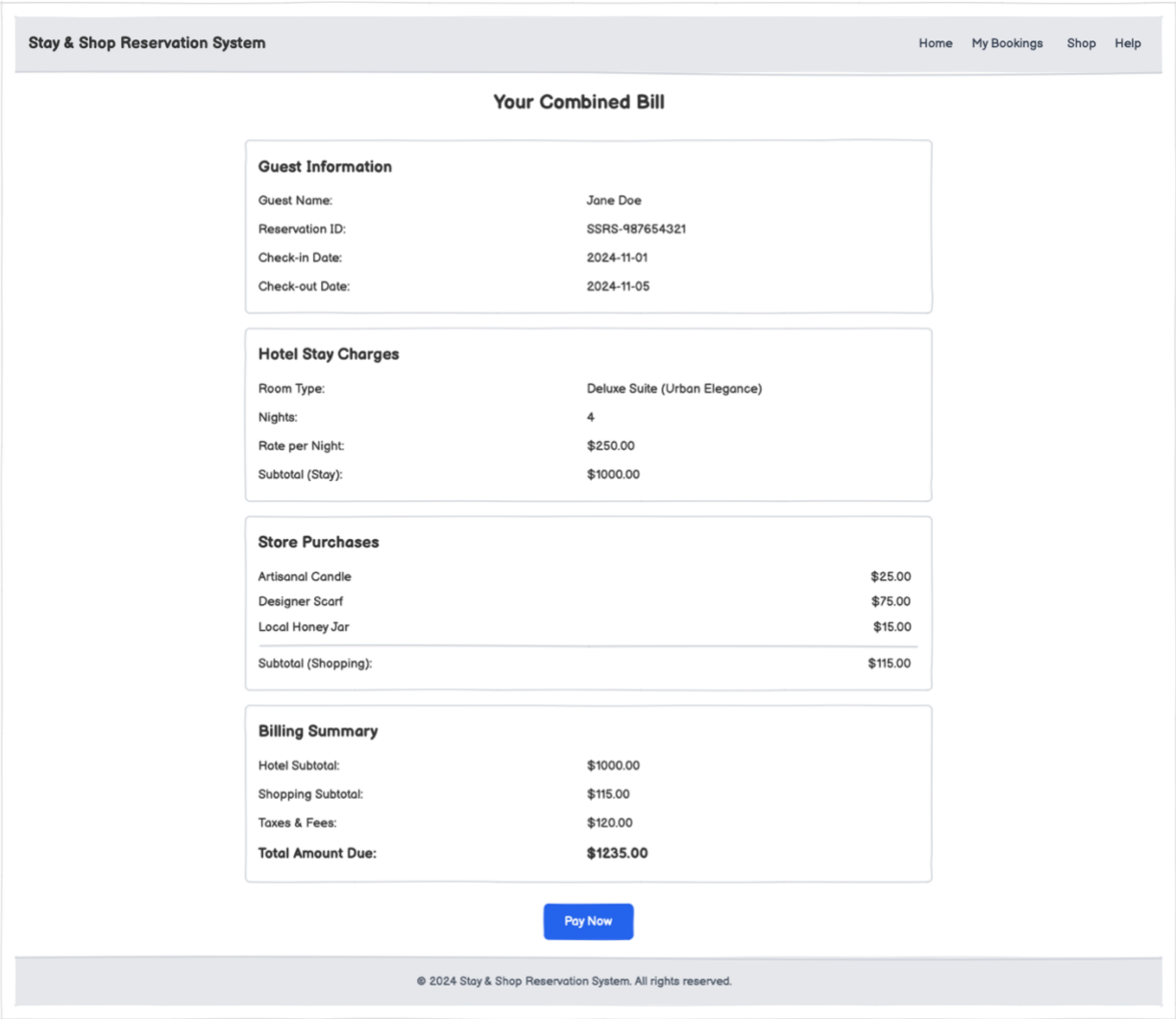

Log in page


Stay & Shop Reservation System
Home
My Bookings
Shop
Help
Log Out
Search for Rooms
Location:
Enter city or destination
Check-in Date:
mm/dd/yyyy
Check-out Date:
mm/dd/yyyy
Number of Guests:
1
Room Type:
Any
Search Rooms
© 2024 Stay & Shop Reservation System. All rights reserved.

**Room Search**




**Hotel shop page**



Stay & Shop Reservation System

Home My Bookings Shop Help

Your Combined Bill

**Guest Information**

| | |
|---|---|
| Guest Name: | Jane Doe |
| Reservation ID: | SSRS-987654321 |
| Check-in Date: | 2024-11-01 |
| Check-out Date: | 2024-11-05 |

**Hotel Stay Charges**

| | |
|---|---|
| Room Type: | Deluxe Suite (Urban Elegance) |
| Nights: | 4 |
| Rate per Night: | $250.00 |
| Subtotal (Stay): | $1000.00 |

**Store Purchases**

| | |
|---|---|
| Artisanal Candle | $25.00 |
| Designer Scarf | $75.00 |
| Local Honey Jar | $15.00 |
| Subtotal (Shopping): | $115.00 |

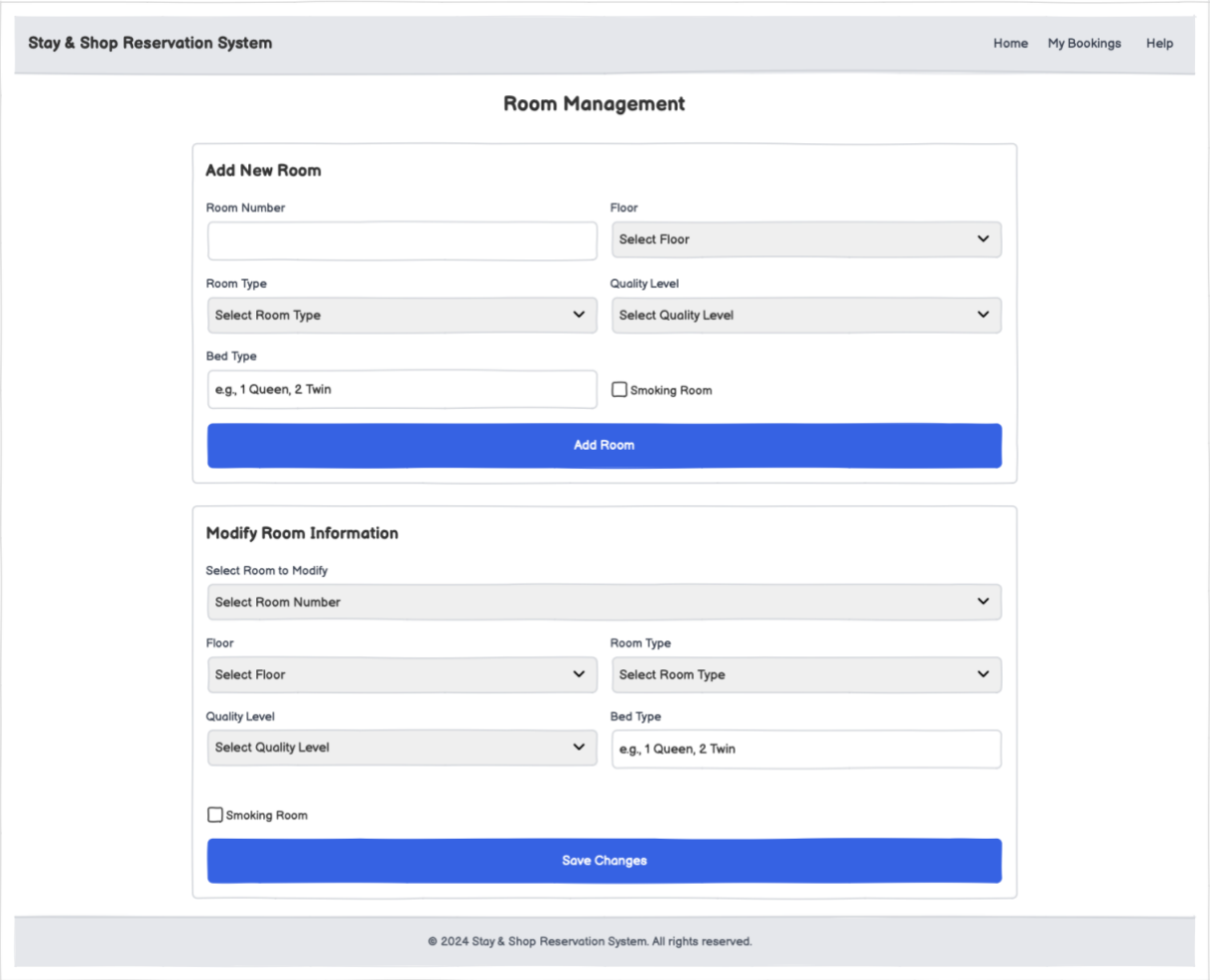
**Billing Summary**

| | |
|---|---|
| Hotel Subtotal: | $1000.00 |
| Shopping Subtotal: | $115.00 |
| Taxes & Fees: | $120.00 |
| **Total Amount Due:** | **$1235.00** |

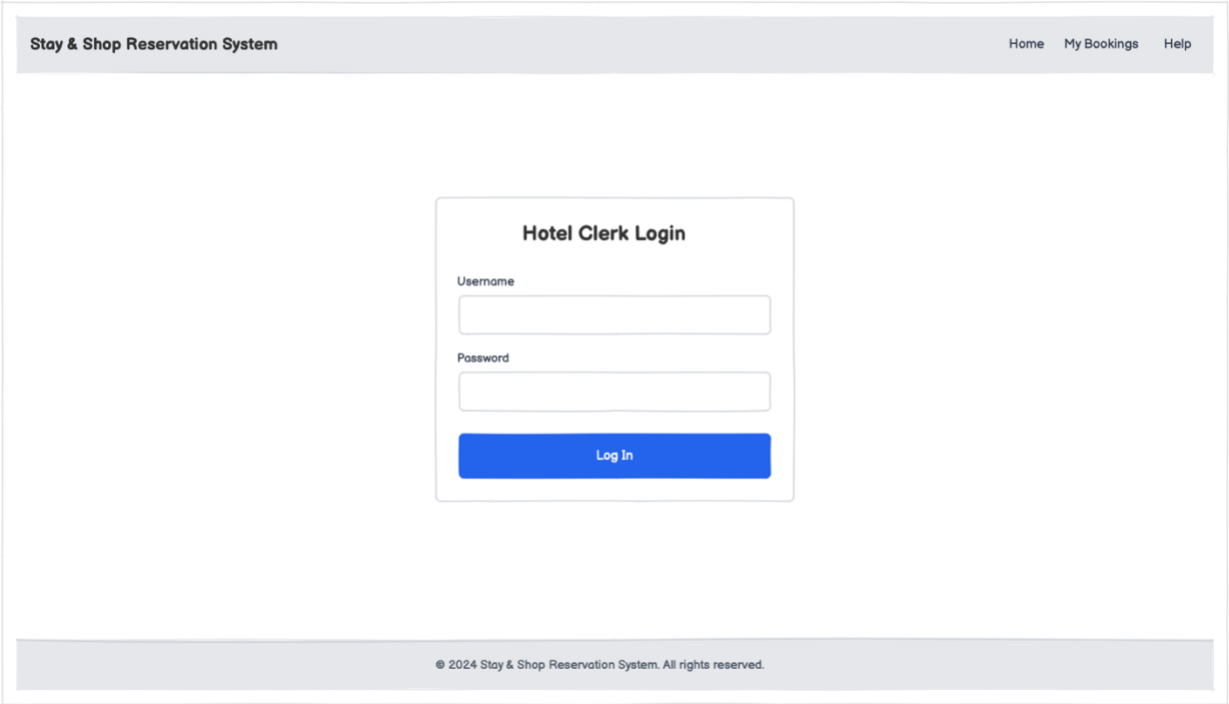
Pay Now

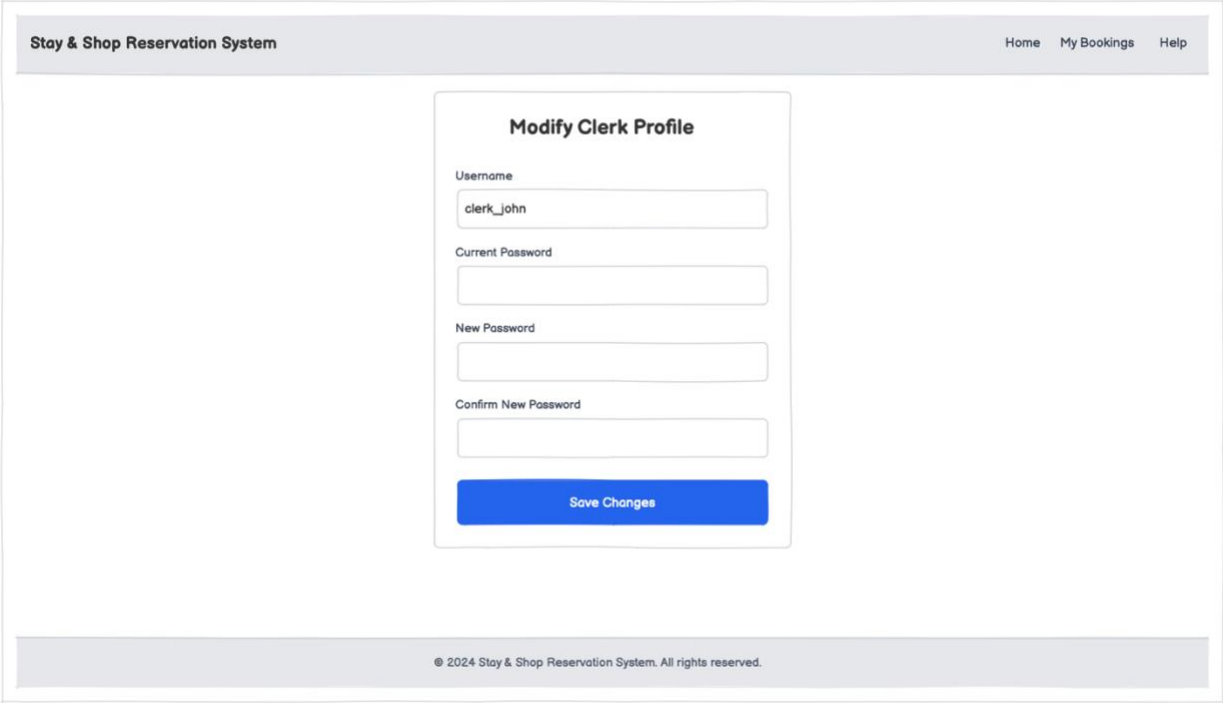
© 2024 Stay & Shop Reservation System. All rights reserved.

**Billing page**

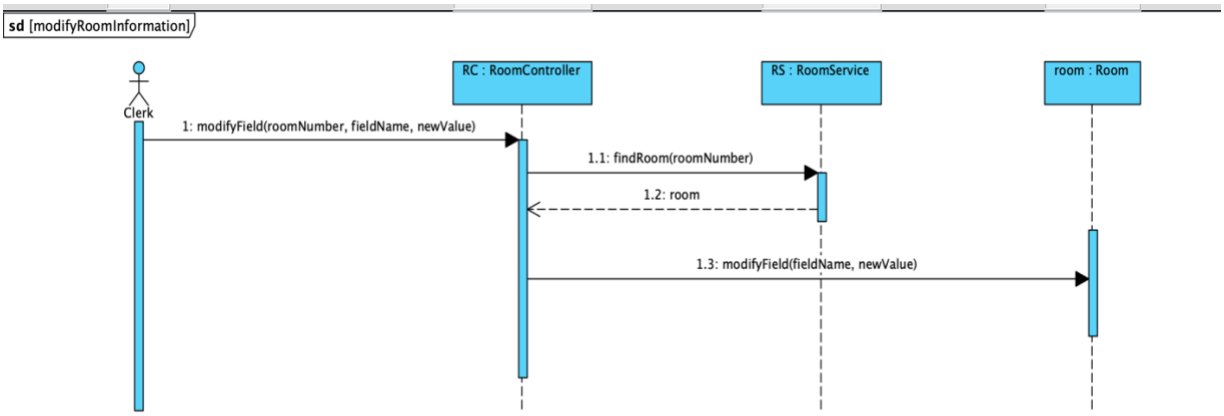



**Clerk room management page**

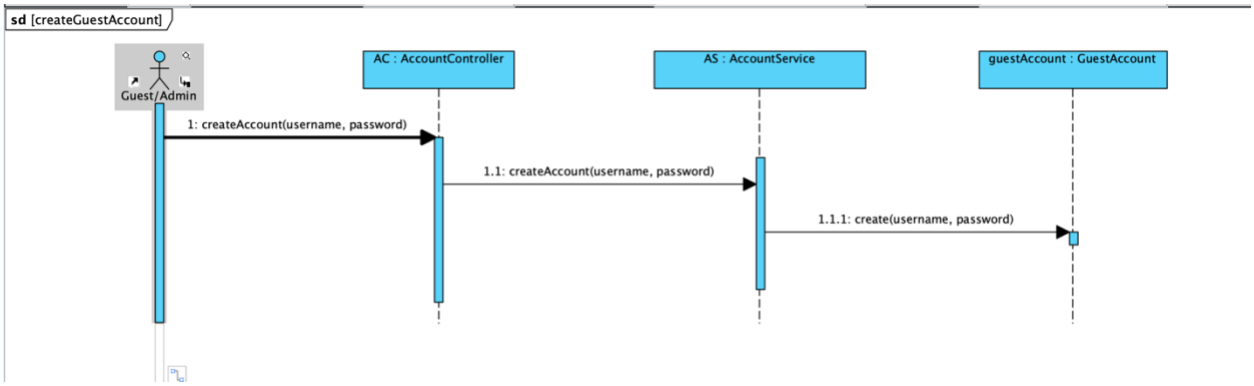


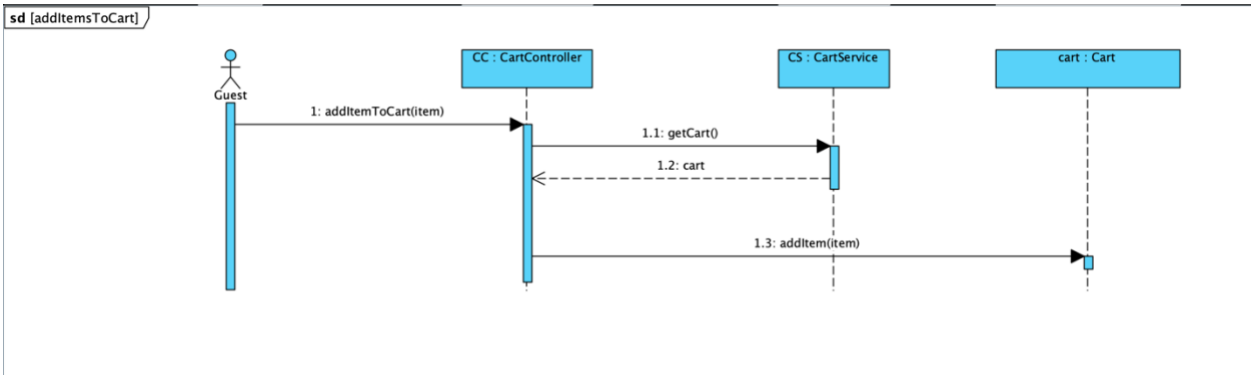

# SEQUENCE DIAGRAMS



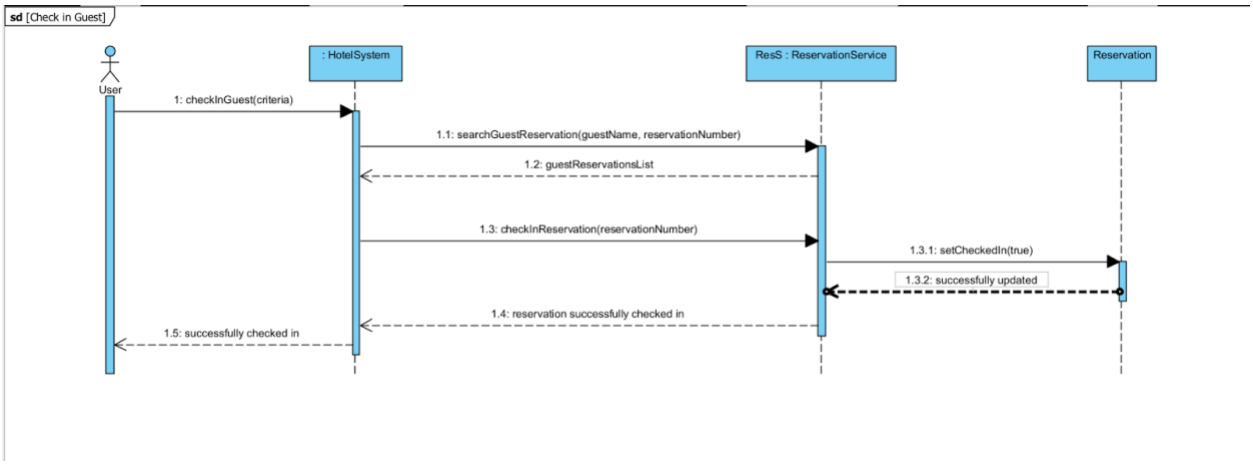

**Modify Room Information (Nolan Schirripa)**



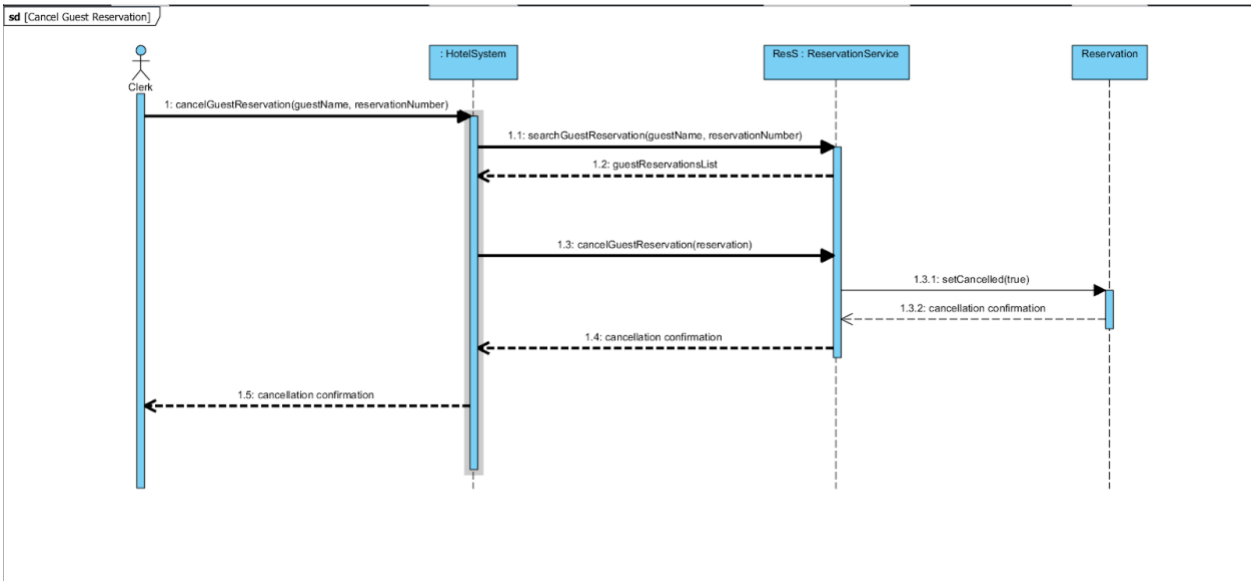

**Create Guest Account (Nolan Schirripa)**



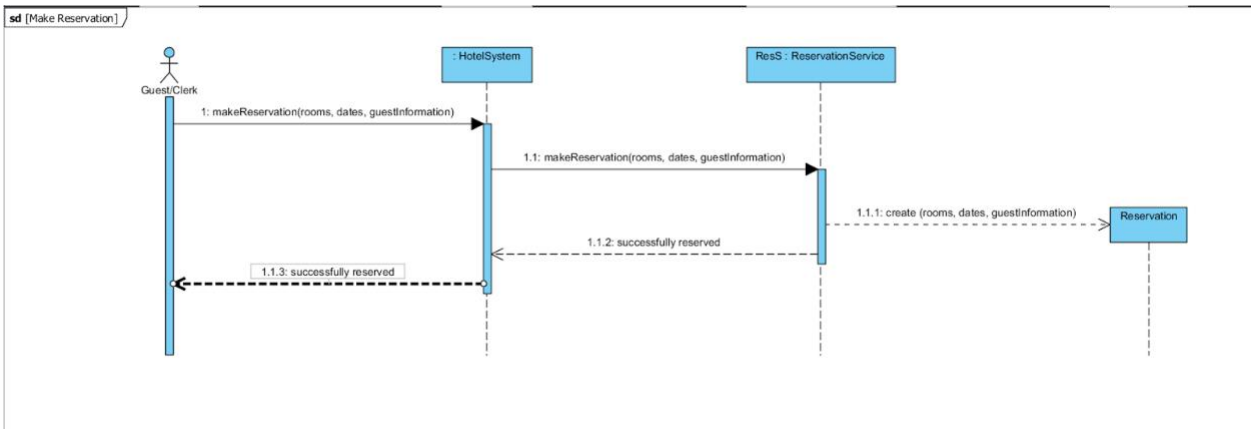

**Add Items to Cart (Nolan Schirripa)**



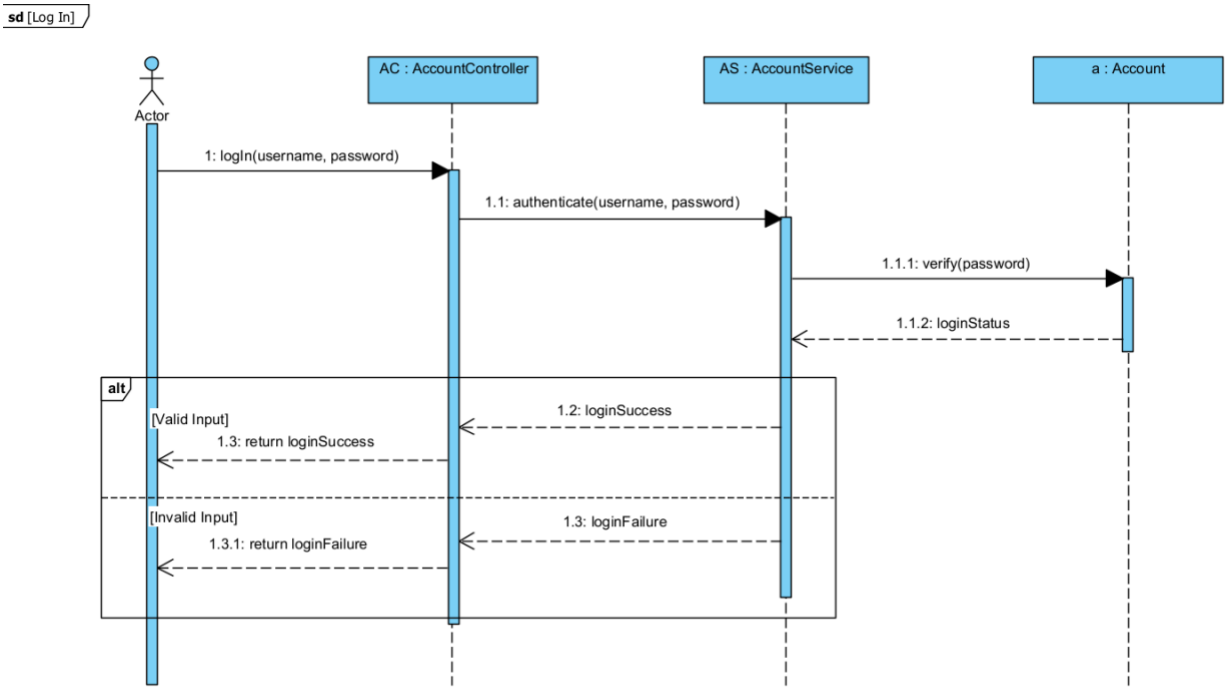

**Check In Guest (Christine Seng)**



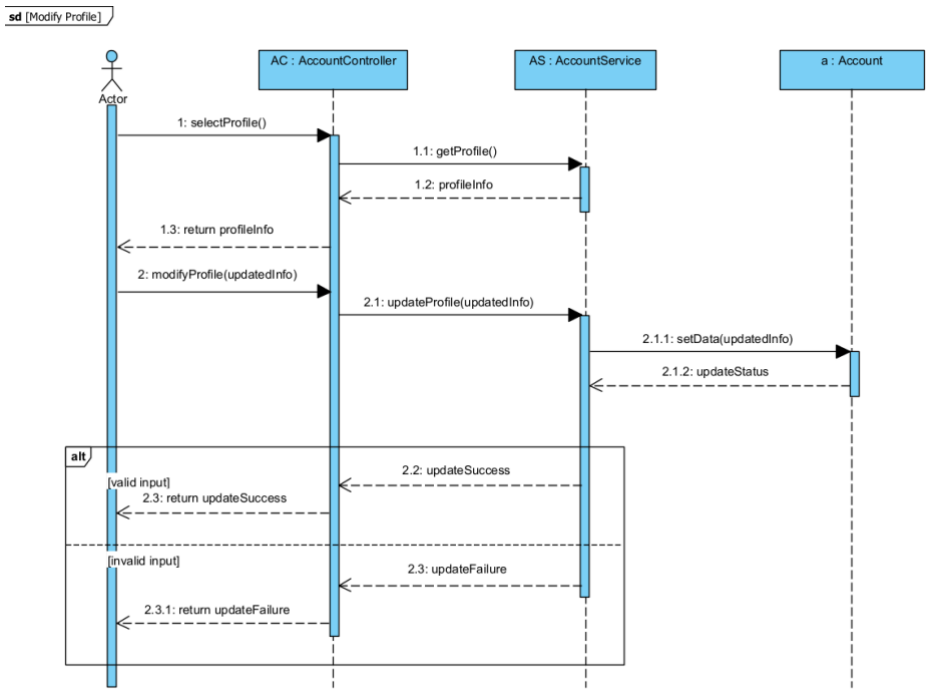

**Cancel Guest Reservation (Christine Seng)**



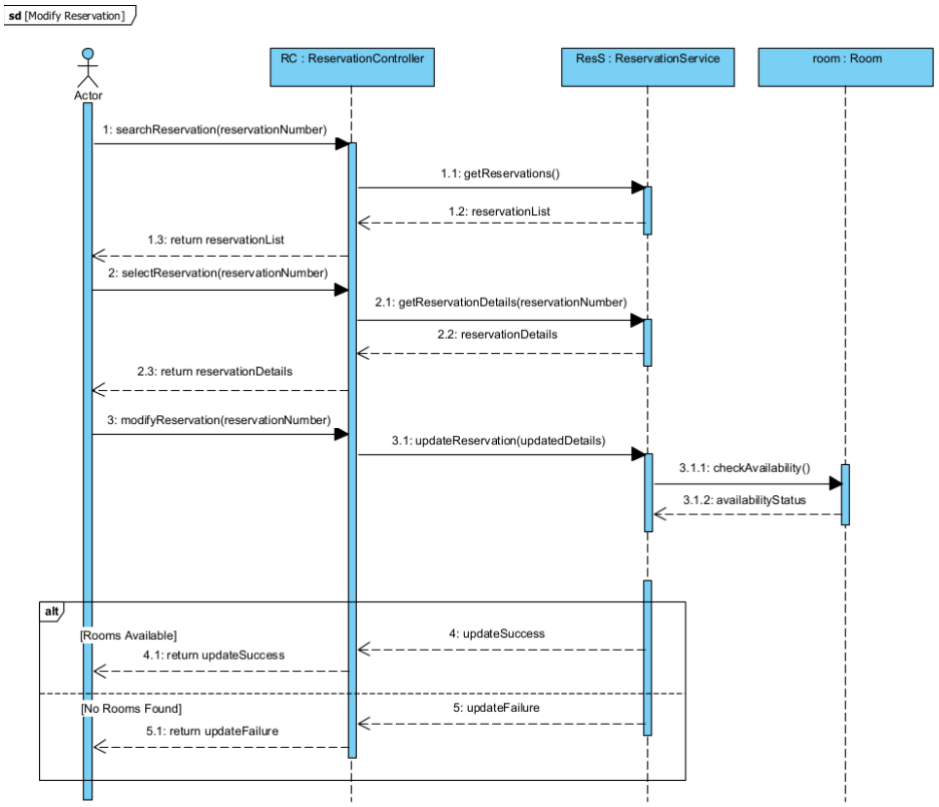

**Make Reservation (Christine Seng)**



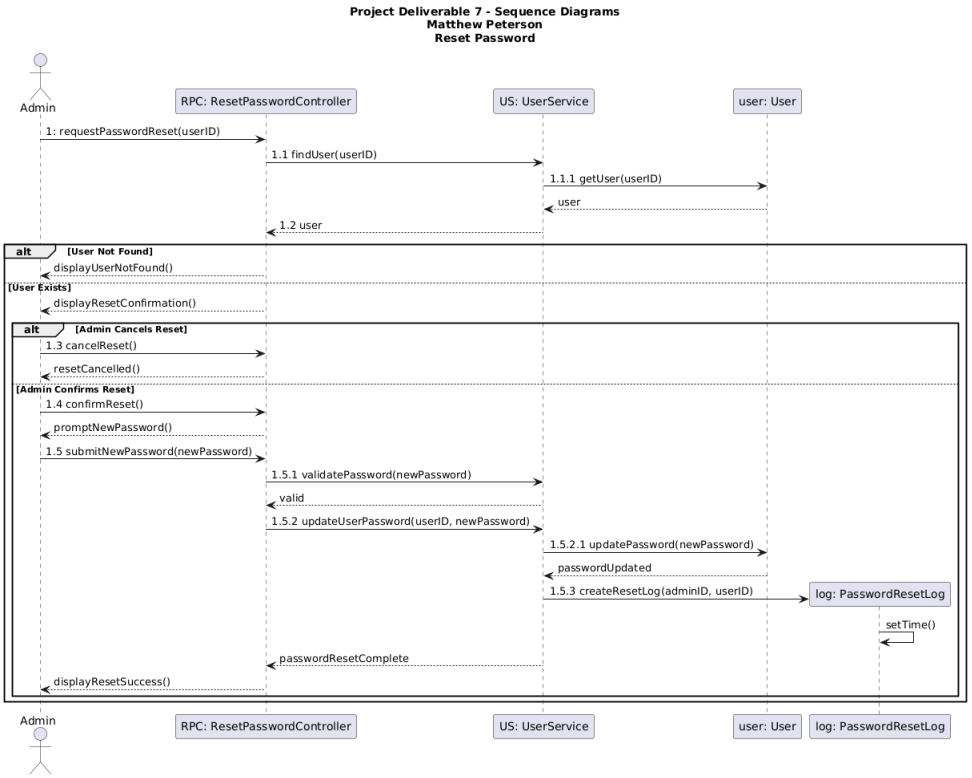

**Log In (Georgia Rushing)**



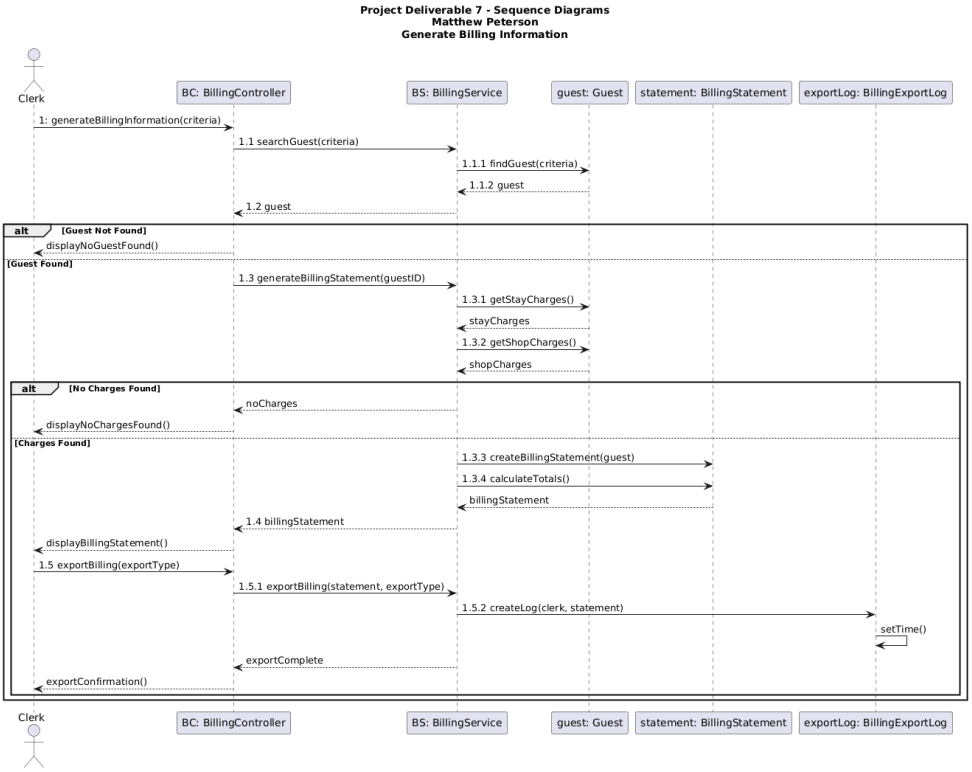

**Modify Profile (Georgia Rushing)**



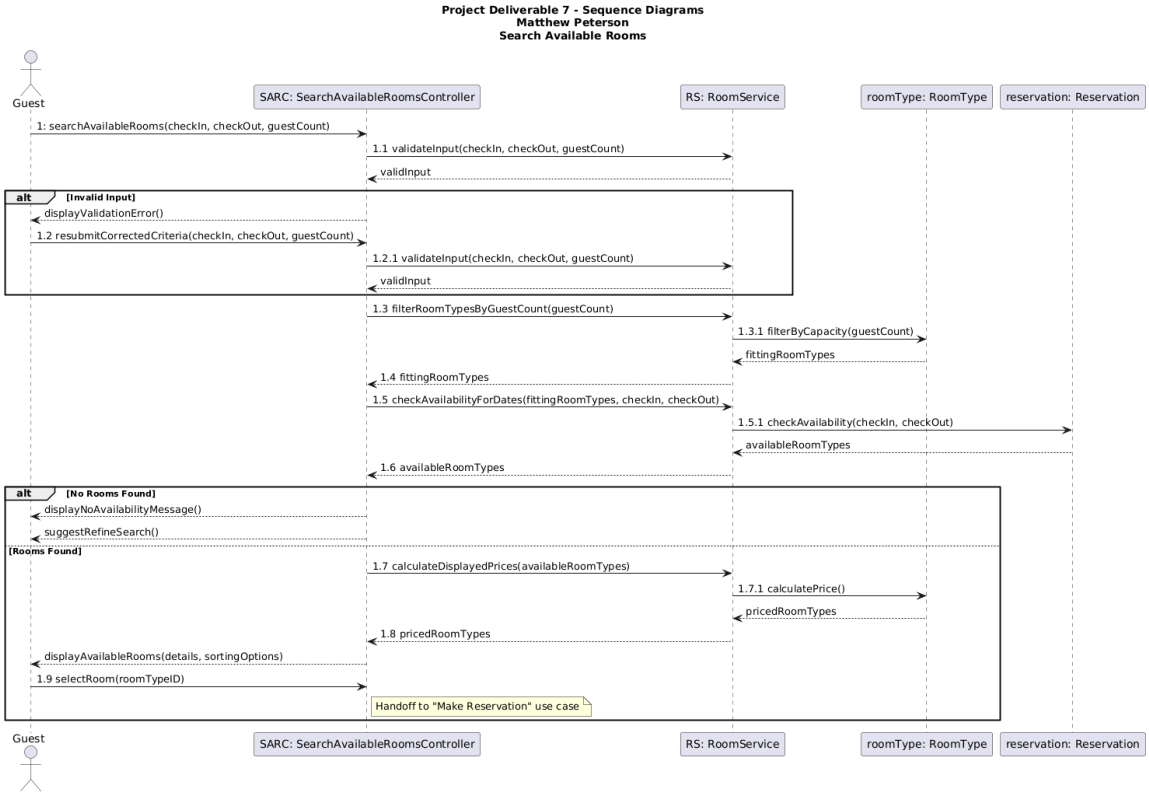

**Modify Reservation (Georgia Rushing)**




## Reset Password (Matthew Peterson)




## Generate Billing Info (Matthew Peterson)




**Search Available Rooms (Matthew Peterson)**